

熟悉嵌入式面试流程，提前通关嵌入式面试

规模较大的公司面试大致流程

- 1.投递简历，筛选简历
- 2.发送笔试
- 3.一面--技术面试（有些公司会有两轮技术面试）
- 4.二面--HR面试（一般是聊对于公司的看法，自己的职业规划等待非技术的东西，这里的问题比较发散）
- 5.发送offer

规模较小的公司

- 1.投递简历，筛选简历
- 2.发送笔试
- 3.技术面试（面试结果满意大概率会当场给你offer）
- 4.HR沟通offer

以诺瓦面试流程举例

一面（技术面试）

- 1.iic地址
- 2.ADC电池采样 电压处理的公式
- 3.怎么设计一个FIFO
- 4.做过多线程的项目 多线程需要注意什么
- 5.指针
- 6.Linux编程 Linux指令
- 7.stm32启动流程
- 8.stm32寄存器的作用
- 9.看你会用示教器，你调过spi吗
- 10.手撕链表翻转
- 11.stm32是大端还是小端？手撕一个判断大端还是小端的程序

岗位测评（一般大部分公司是在笔试前面进行测评）

- 1.逻辑推断题
- 2.心理题

二面（综合面）

- 1、自我介绍
- 2、在比赛和项目中充当什么角色，你作为负责人是负责哪些地方，如何协调其他组员的
- 3、你在本科和研究生学习中有什么不同
- 4、是什么契机让你参加这些比赛
- 5、选一个你印象最深的比赛

三面（HR面）

- 1.聊对于公司的看法
- 2.自己的职业规划
- 3.薪资的要求
- 4.家人的看法

5.大学生涯的评价
等待

发送offer

1. USART 和 UART 的区别？

答:1.同步与异步通信：最主要的区别在于同步和异步通信的支持。**USART** 支持同步（**Synchronous**）和异步（**Asynchronous**）两种通信模式，而 **UART** 仅支持异步通信模式。

2.时钟信号：**USART** 在同步通信模式下需要一个外部的时钟信号来同步数据传输，而在异步通信模式下不需要时钟信号。**UART** 只能通过异步方式进行数据传输，因此不需要时钟信号。

3.数据帧格式：**USART** 提供了更多的数据帧格式选项，包括数据位数、停止位数、奇偶校验等，并且可以通过配置来实现更高的数据传输速率。**UART** 则通常仅支持常见的数据帧格式，如 8 个数据位、1 个停止位。

4.错误检测：在 **USART** 中，由于同步通信的支持，可以进行更强大的错误检测，如使用差错检测码（**CRC**）来保证数据的完整性。而在 **UART** 中，由于仅支持异步通信，错误检测的能力相对较弱。

5.**USART** 和 **UART** 在硬件电路层面上可能会存在些许差异，但它们的基本功能都是提供串行通信的能力。在实际应用中，选择 **USART** 还是 **UART** 取决于具体需求和硬件支持情况。

2. UART 优缺点？

答：优点： 简单易用：**UART** 是一种简单的串行通信接口，容易实现和使用。它只需要两条信号线（发送线和接收线），不需要复杂的时钟同步机制，适合于嵌入式系统和普通计算设备。

广泛兼容性：**UART** 是一种标准接口，广泛用于各种设备之间的串行通信，例如计算机与外部设备（如传感器、显示屏、**GPS** 模块等）之间的数据交互。

低成本：由于 **UART** 的简单性，相对于其他复杂的串行通信接口，它的硬件成本较低，使得它在成本敏感的应用中非常有吸引力。

通信距离：**UART** 的异步传输方式使得它在通信距离上具有一定的优势。它可以通过电平转换器和适当的线缆来实现长距离的通信。

缺点：速度限制：由于 **UART** 是异步通信，其传输速度受到一些限制。较高的数据传输速率可能会导致数据精度下降或信号失真。相比之下，同步通信接口（如 **SPI** 和 **I2C**）在高速数据传输方面更有优势。

无错恢复机制：**UART** 缺乏强大的错误检测和纠正机制。它没有内置的差错检测码（如 **CRC**），因此对于数据完整性的要求较高的应用，**UART** 可能不太适合。

只支持点对点通信：**UART** 是一种点对点通信协议，每个 **UART** 接口只能实现两个设备之间的直接通信。如果需要多个设备进行通信，可能需要额外的复用器或其他解决方案。

3. IIC 三种信号

答：时钟信号，数据信号、地址

4. 画 IIC 时序图

5. 数据传输流程

6. IIC 工作速率

答：标准模式、快速模式、高速模式

7. 怎样用 IO 口模拟 IIC?

答：IO 口的设置通常为开漏输出模式和开漏输入模式，并配合外部上拉电阻的使用。开漏输出模式下，IO 口可以输出低电平或高阻态（高电平时，IO 口处于三态，即不输出电平）。这样可以与其他设备的开漏输入端口连接，形成一个共享的总线结构，IO 口输出对应的时序即可进行通信

8. IIC 如何进行读写操作?

答：七位地址位后设置 0/1 进行设置读写

9. IIC 总线仲裁机制是怎么样的?

答：当多人设备同时请求访问总线时，主设备会发送一个仲裁信号，这个信号包含一个特殊的比特位，称为“仲裁位”。所有设备都会监听这个仲裁位，如果发现自己发送的地址与仲裁信号中的地址不匹配，就会放弃访问总线，等待下一次机会

10. IIC 电气特性，为什么要使用上拉电阻，IIC 有哪些模式，传输速率如何。IIC 传输中可能会遇到哪些错误?

答：使用上拉电阻的目的是确保在 SDA 和 SCL 线上的电平可以稳定地保持在高电平状态。当总线上没有任何设备向下拉低 SDA 线时，上拉电阻将提供一个足够的电流，使得 SDA 线上的电压保持在高电平。

标准模式（时钟频率最高为 100 kHz）、快速模式（最高可达 400 kHz）、高速模式（最高可达 3.4 MHz）

总线冲突、应答错误、超时、地址错误、校验错误

11. 介绍 SPI

答：我主要接触的 spi 有两种，一个三线 spi、一个四线 spi，有时钟信号 SCLK、从设备输入 MOSI、主设备输入 MISO、片选 CS，支持全双工通信

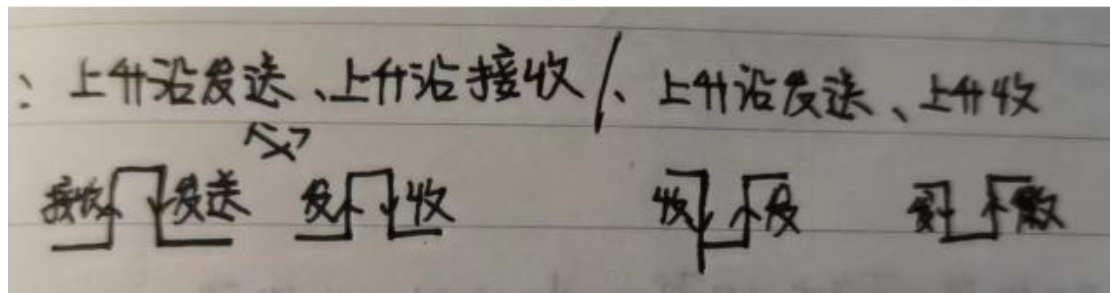
12. 最大传输速率受什么影响?

答：主设备和从设备、时钟频率

13. 四种模式及其对应的时序图

答

:



14. SPI 如何读写？

答： 读操作：

MISO, 接收单片机发来的数据(指令)，发完指令，发 24 位地址。

MOSI, 单片机接收数据。

15. 函数指针和指针函数的区别

答：函数指针是指向函数的指针变量。指针函数是一个返回指针的函数。它是一个函数，其返回类型是一个指针类型。

16. 数组和指针的区别

答：数组是一组连续的内存空间，用于存储相同类型的元素。指针是一个变量，存储着内存地址，可以指向任何类型的数据。

17. 引用和指针相比有什么不用

答：1. 引用必须在声明时进行初始化，并且一旦初始化后，它将永远引用同一个对象。而指针可以被赋予空值(NULL)，表示不指向任何有效的内存地址。

2. 引用一旦被初始化，就不能再引用其他对象了，因此不能被修改为引用另一个对象。指针可以通过重新赋值来指向不同的对象。

3. 引用不需要分配内存，它只是已存在对象的别名。指针需要分配内存来存储指针变量本身，并且可以通过指针动态分配和释放内存。

4. 引用不需要分配内存，它只是已存在对象的别名。指针需要分配内存来存储指针变量本身，并且可以通过指针动态分配和释放内存。

18. 全局变量和局部变量的储存方式

答：全局变量通常存储在静态数据区或全局数据区

局部存储在栈(stack)上

19. C 中的内存分配方式有几种

答：1. 静态存储区：静态存储区是在程序运行期间就分配并一直存在的内存区域。它包括存储全局变量和静态变量的内存空间。

静态存储区在程序启动时被分配，直到程序结束才释放。

2. 栈存储区：栈存储区用于管理函数的局部变量和函数调用的上下文信息。在每个函数调用时，会为该函数分配一块栈帧，

其中包含参数、局部变量和返回地址等信息。栈是一种后进先出(LIFO)的数据结构，因此栈中的内存块按照函数调用的顺序分配和释放。

3.堆存储区：堆存储区是由程序员手动分配和释放的动态内存区域。它提供了一种灵活的内存分配方式，可以在程序运行时动态地申请和释放内存块。堆存储区通常用于存储动态数据结构，如动态数组、链表和对象等。通过函数如 `malloc()`、`calloc()` 和 `realloc()` 可以从堆上分配内存，并通过 `free()` 函数释放已分配的内存。

4. 常量存储区：常量存储区用于存储常量数据，如字符串常量和全局常量变量。这些数据是在编译时就确定的，并且存储在只读的内存区域中。

20. IIC 通信方式

答：半双工、数据线 `sda`、时钟线 `scl`

21. 中断概念

答：中断是计算机系统中的一种机制，用于处理来自外部设备或内部事件的异步事件

22. 中断过程

答：发生事件->请求中断 ->响应中断 ->中断向量表索引 ->中断服务程序执行 ->结束恢复现场

23. 中断能否传参

答：不能 一般都是通过全局变量方式。不过为了避免中断函数重入，我们可以在进入中断服务函数之后将中断关闭，处理完后再开中断。 中断函数是收到中断信号自动跳转的，并非通过函数调用，什么时候运行是预先不确定的，所以不支持传参数。

24. IIC 数据帧格式

答：start 信号 ->7 位地址和读写位 ->应答位 ->数据 ->应答位 ->stop 信号

25. UART 通信波特率

答：波特率（Baud Rate）是衡量 UART 通信速度的单位，表示每秒传输的比特数。 较低的波特率适用于长距离通信或者噪声环境下，而较高的波特率适用于短距离通信或者要求更高的实时性。

26. UART 有几根线

答：两根传输线 `tx` 和接收线 `rx` `gnd`、`vcc`

27. UART 同步还是异步

答：异步通信

28. UART 数据帧格式

答：起始位、数据位、校验位、停止位

29. IIC 应答数据方式

答：接收方成功接收数据，拉低 **sda** 发送低电平信号，没有接收成功就维持 **sda** 高电平

30. SPI 通信波特率

答：stm32f103 波特率最高是 18mhz

31. ADC 位数精度如何计算

答：一个 12 位的 ADC 可以将输入的模拟信号转换为一个 12 位的数字值，即有 4096 (2^{12}) 个离散的量化水平。

位数精度的计算可以通过以下公式进行：

精度 (in bits) = $\log_2$ (量化水平数)

以 12 位的 ADC 为例，它具有 $2^{12} = 4096$ 个离散的量化水平。因此，它的位数精度可以通过如下公式计算：

精度 (in bits) = $\log_2 (4096) = 12$ bits

对于一个 12 位 ADC，如果其测量范围为 0V 到 3.3V，则最小分辨步进值为：

最小分辨步进值 = $3.3V / (2^{12}) \approx 0.000805V$

32. GPIO 有几种输出模式

答：推挽输出、开漏输出、复用开漏输出、复用推挽输出
一些引脚有 pwm 脉冲宽度调制输出、模拟输出

33. 开漏输出和推挽输出的区别

答：输出电平：

开漏输出：在开漏输出模式下，引脚可以拉低（连接到地/GND），但不能直接提供高电平输出。引脚相当于一个开关，只能将引脚拉低和保持高阻态。要实现高电平，需要外部上拉电阻将引脚拉高到高电平。

推挽输出：在推挽输出模式下，引脚可以提供高电平和低电平输出。当输出高电平时，引脚与高电压连接；当输出低电平时，引脚与地（GND）连接。

输出能力：

开漏输出：开漏输出通常具有较大的输出电流能力，可以驱动较高的负载电流。它经常用于多个引脚共享一个总线/线路的情况，如 I2C 总线。

推挽输出：推挽输出可以提供较大的输出电流。它适用于直接驱动负载的情况，如 LED、电机等。

上拉电阻：

开漏输出：开漏输出模式下，引脚需要外部上拉电阻来将引脚拉高到高电平。这是因为在开漏输出状态下，引脚只负责拉低（连接到地），不直接提供高电平输出。

推挽输出：推挽输出模式下，引脚可以直接提供高电平输出，不需要外部上拉电阻。

多路复用能力：

开漏输出：开漏输出可以用于实现多个引脚共享一个总线/线路的情况，如 I2C 总线。多个开漏输出引脚可以通过外部上拉电阻和总线上的设备进行逻辑连接。

推挽输出：推挽输出引脚通常适合单独驱动独立的设备或信号线

34. 为什么栈从上往下生长？

答：堆是往上涨的，栈是往下涨的，假如栈往上涨的话，那么要确定堆和栈的分界线，每一个程序的堆和栈都一样，所以这个分界线不好确定，所以一个往上涨，一个往下长可以最大化的利用内存的空间。

1. 哪些函数不能在中断中使用？

答：动态内存分配函数：例如 `malloc()`、`free()` 等函数，因为在中断中动态分配和释放内存可能导致内存碎片问题，甚至造成内存泄漏。

阻塞函数：包括像 `sleep()`、`delay()` 等需要等待一段时间的函数，因为在中断服务程序中阻塞会影响系统的实时性。

信号量操作函数：如 `sem_wait()`、`sem_post()` 等，因为这些函数可能会导致死锁或竞争条件。

文件操作函数：例如 `fopen()`、`fclose()` 等，因为在中断服务程序中访问文件可能会引起不可预测的结果。

2. 什么叫不可重入函数？

答：不可重入函数是指在同一时间只能被一个任务或线程执行的函数，不可重入函数会使用一些全局变量或静态变量来保存状态信息，如果在该函数执行过程中被中断，并在中断处理程序或其他任务中再次调用该函数，那么全局状态可能会被破坏，导致函数无法正确执行。

3. 如果在一个中断里面有一个变量，其他线程里面需要读取这个变量要注意什么？

答：我会用 `volatile` 对这个变量进行修饰，在使用这个变量的其他线程中创建一个互斥量，获得锁读取这个变量的，然后释放锁。

```
volatile int sharedVariable = 0;
```

中断

```
{  
    xSemaphoreTakeFromISR(xSemaphore, NULL);  
    sharedVariable = 42;  
    xSemaphoreGiveFromISR(xSemaphore, NULL);  
}
```

4. 定时器 PWM？

答：周期由定时器的计数值决定。占空比表示高电平状态（信号为 1）在一个周期内所占的时间比例，它决定了 PWM 信号的电平平均值。当定时器的计数值小于占空比乘以周期时，PWM 信号处于高电平状态。当计数值大于占空比乘以周期时，PWM 信号处于低电平状态。

5. 定时器有几种用法

答：1.最常用的就是延迟 `delay`

2.定时中断，`tick` 中断

3.PWM 输出控制电机的

4.计时计数

6. `const` 关键字？

答：1.将变量修饰成常量，放在 `flash` 上

2.修饰函数参数，防止其他函数调用这个函数的时候，把里面的值修改了。

3.修饰指针 `const int *p`、`int *const p`

7. static 关键字?

答: 1.static 修饰的变量放在静态存储区, 只初始化一次, 一般是写状态机的时候, 一个任务时间消耗的时间比较大, 就可以分段进行
2. static 修饰的函数, 让这个函数只在这个文件里面生效, 其他文件要使用这个函数只能使用我暴露出去的结构体, 或者使用 **extern** 去调用, 更加安全

8. volatile 关键字?

答: 1.使用 **volatile** 一般就是使用物理内存的时候去告诉 **cpu** 说不可以去优化他, 这样确保每次都是最新的值
2.调试的时候, 有些操作需要记录下来, 使用 **volatile** 修饰, 可以不会被 **cpu** 优化了
3.常见的就是不同线程中的通信, 这样可以保证变量的准确性
4.中断里面用的变量一般都建议用 **volatile** 去修饰, 防止被其他的中断改了。

9. 数组和链表的区别?

答: 1.数组要在块连续的空间上, 链表由节点组成。
2.链表的内存开销比较大
3.数组可以快速访问任意位置的元素, 链表只能按顺序访问

10. 通用寄存器包括 R0~R15, 可以分为具体哪三类?

答: R0-R3(传入函数参数, 传出函数返回值), R4-R12(存局部变量), R13(SP 栈指针), R14(LR 寄存器), R15(PC 寄存器)

11. 如何对绝对地址 0x100000 赋值?

答: c 语言

```
volatile uint32_t* address = (volatile uint32_t*)0x100000;  
*address = value;
```

汇编

```
LDR R0, =0x100000    ; 将地址 0x100000 加载到寄存器 R0  
LDR R1, =value       ; 将 value 加载到寄存器 R1  
STR R1, [R0]         ; 将 R1 的值存储到 R0 指向的地址
```

12. stm32 寄存器详解

答: R0-R3: 传入函数参数, 传出函数返回值; R4-R12: 存放函数的局部变量; R13: SP 栈指针; R14(LR): 保存返回地址; R15(PC): 用于存储下条指令的地址

13. 什么是交叉编译?

答: 交叉编译是指使用一台计算机来编译另一种不同架构的目标平台上的程序。

C/C++经典面试题

面试题 1：变量的声明和定义有什么区别

为变量分配地址和存储空间的称为定义，不分配地址的称为声明。一个变量可以在多个地方声明，但是只在一个地方定义。加入 **extern** 修饰的是变量的声明，说明此变量将在文件以外或在文件后面部分定义。

说明：很多时候一个变量，只是声明不分配内存空间，直到具体使用时才初始化，分配内存空间，如外部变量。

面试题 2：写出 bool 、int、 float、指针变量与“零值”比较的 if 语句

bool 型数据：

```
if( flag )
{
    A;
}
else
{
    B;
}
```

int 型数据：

```
if( 0 != flag )
{
    A;
}
else
{
    B;
}
```

指针型数：

```
if( NULL == flag )
{
    A;
}
else
{
    B;
}
```

float 型数据：

```
if ( ( flag >= NORM ) && ( flag <= NORM ) )
{
    A;
}
```

```
}
```

注意：应特别注意在 int、指针型变量和“零值”比较的时候，把“零值”放在左边，这样当把“==”误写成“=”时，编译器可以报错，否则这种逻辑错误不容易发现，并且可能导致很严重的后果。

面试题 3：sizeof 和 strlen 的区别

sizeof 和 strlen 有以下区别：

- ❑ sizeof 是一个操作符，strlen 是库函数。
- ❑ sizeof 的参数可以是数据的类型，也可以是变量，而 strlen 只能以结尾为‘\0’的字符串作参数。
- ❑ 编译器在编译时就计算出了 sizeof 的结果。而 strlen 函数必须在运行时才能计算出来。并且 sizeof 计算的是数据类型占内存的大小，而 strlen 计算的是字符串实际的长度。
- ❑ 数组做 sizeof 的参数不退化，传递给 strlen 就退化为指针了。

注意：有些是操作符看起来像是函数，而有些函数名看起来又像操作符，这类容易混淆的名称一定要加以区分，否则遇到数组名这类特殊数据类型作参数时就很容易出错。最容易混淆为函数的操作符就是 sizeof。

面试题 4：C 语言的关键字 static 和 C++ 的关键字 static 有什么区别

在 C 中 static 用来修饰局部静态变量和外部静态变量、函数。而 C++ 中除了上述功能外，还用来定义类的成员变量和函数。即静态成员和静态成员函数。

注意：编程时 static 的记忆性，和全局性的特点可以让在不同时期调用的函数进行通信，传递信息，而 C++ 的静态成员则可以在多个对象实例间进行通信，传递信息。

面试题 5：C 中的 malloc 和 C++ 中的 new 有什么区别

malloc 和 new 有以下不同：

- (1) new、delete 是操作符，可以重载，只能在 C++ 中使用。
- (2) malloc、free 是函数，可以覆盖，C、C++ 中都可以使用。
- (3) new 可以调用对象的构造函数，对应的 delete 调用相应的析构函数。
- (4) malloc 仅仅分配内存，free 仅仅回收内存，并不执行构造和析构函数
- (5) new、delete 返回的是某种数据类型指针，malloc、free 返回的是 void 指针。

注意：malloc 申请的内存空间要用 free 释放，而 new 申请的内存空间要用 delete 释放，不要混用。因为两者实现的机理不同。

面试题 6：写一个“标准”宏 MIN

```
#define min(a,b)((a)<=(b)?(a):(b))
```

注意：在调用时一定要注意这个宏定义的副作用，如下调用：

```
((++*p)<=(x)?(++*p):(x))。
```

p 指针就自加了两次，违背了 MIN 的本意。

面试题 7: 一个指针可以是 volatile 吗

可以, 因为指针和普通变量一样, 有时也有变化程序的不可控性。常见例: 子中断服务子程序修改一个指向一个 buffer 的指针时, 必须用 volatile 来修饰这个指针。

说明: 指针是一种普通的变量, 从访问上没有什么不同于其他变量的特性。其保存的数值是个整型数据, 和整型变量不同的是, 这个整型数据指向的是一段内存地址。

面试题 8: a 和 &a 有什么区别

请写出以下代码的打印结果, 主要目的是考察 a 和 &a 的区别。

```
#include<stdio.h>
void main( void )
{
    int a[5]={1,2,3,4,5};
    int *ptr=(int *)&a+1;

    printf("%d,%d",*(a+1),*(ptr-1));
    return;
}
```

输出结果: 2, 5。

注意: 数组名 a 可以作数组的首地址, 而 &a 是数组的指针。思考, 将原式的 `int *ptr=(int *)&a+1;` 改为 `int *ptr=(int *)a+1;` 时输出结果将是什么呢?

面试题 9: 简述 C、C++ 程序编译的内存分配情况

C、C++ 中内存分配方式可以分为三种:

(1) 从静态存储区域分配:

内存在程序编译时就已经分配好, 这块内存在程序的整个运行期间都存在。速度快、不容易出错, 因为有系统会善后。例如全局变量, static 变量等。

(2) 在栈上分配:

在执行函数时, 函数内局部变量的存储单元都在栈上创建, 函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集中, 效率很高, 但是分配的内存容量有限。

(3) 从堆上分配:

即动态内存分配。程序在运行的时候用 malloc 或 new 申请任意大小的内存, 程序员自己负责在何时用 free 或 delete 释放内存。动态内存的生存期由程序员决定, 使用非常灵活。如果在堆上分配了空间, 就有责任回收它, 否则运行的程序会出现内存泄漏, 另外频繁地分配和释放不同大小的堆空间将会产生堆内碎块。

一个 C、C++ 程序编译时内存分为 5 大存储区: 堆区、栈区、全局区、文字常量区、程序代码区。

面试题 10：简述 strcpy、sprintf 与 memcpy 的区别

三者主要有以下不同之处：

(1) 操作对象不同，strcpy 的两个操作对象均为字符串，sprintf 的操作源对象可以是多种数据类型，目的操作对象是字符串，memcpy 的两个对象就是两个任意可操作的内存地址，并不限于何种数据类型。

(2) 执行效率不同，memcpy 最高，strcpy 次之，sprintf 的效率最低。

(3) 实现功能不同，strcpy 主要实现字符串变量间的拷贝，sprintf 主要实现其他数据类型格式到字符串的转化，memcpy 主要是内存块间的拷贝。

说明：strcpy、sprintf 与 memcpy 都可以实现拷贝的功能，但是针对的对象不同，根据实际需求，来选择合适的函数实现拷贝功能。

面试题 11：设置地址为 0x67a9 的整型变量的值为 0xaa66

```
int *ptr;  
ptr = (int *)0x67a9;  
*ptr = 0xaa66;
```

说明：这道题就是强制类型转换的典型例子，无论在什么平台地址长度和整型数据的长度是一样的，即一个整型数据可以强制转换成地址指针类型，只要有意义即可。

面试题 12：面向对象的三大特征

面向对象的三大特征是封装性、继承性和多态性：

- ❑ 封装性：将客观事物抽象成类，每个类对自身的数据和方法实行 protection（private，protected，public）。
- ❑ 继承性：广义的继承有三种实现形式：实现继承（使用基类的属性和方法而无需额外编码的能力）、可视继承（子窗体使用父窗体的外观和实现代码）、接口继承（仅使用属性和方法，实现滞后到子类实现）。
- ❑ 多态性：是将父类对象设置成为和一个或多个它的子对象相等的技术。用子类对象给父类对象赋值之后，父类对象就可以根据当前赋值给它的子对象的特性以不同的方式运作。

说明：面向对象的三个特征是实现面向对象技术的关键，每一个特征的相关技术都非常的复杂，程序员应该多看、多练。

面试题 13：C++的空类有哪些成员函数

- ❑ 缺省构造函数。
- ❑ 缺省拷贝构造函数。
- ❑ 缺省析构函数。
- ❑ 缺省赋值运算符。
- ❑ 缺省取址运算符。
- ❑ 缺省取址运算符 const。

注意：有些书上只是简单的介绍了前四个函数。没有提及后面这两个函数。但后面这两个函数也是空类的默认函数。另外需要注意的是，只有当实际使用这些函数的时候，编译器才会去定义它们。

面试题 14：谈谈你对拷贝构造函数和赋值运算符的认识

拷贝构造函数和赋值运算符重载有以下两个不同之处：

(1) 拷贝构造函数生成新的类对象，而赋值运算符不能。

(2) 由于拷贝构造函数是直接构造一个新的类对象，所以在初始化这个对象之前不用检验源对象是否和新建对象相同。而赋值运算符则需要这个操作，另外赋值运算中如果原来的对象中有内存分配要先把内存释放掉

注意：当有类中有指针类型的成员变量时，一定要重写拷贝构造函数和赋值运算符，不要使用默认的。

面试题 15：用 C++ 设计一个不能被继承的类

```
template <typename T> class A
{
    friend T;
private:
    A() {}
    ~A() {}
};

class B : virtual public A<B>
{
public:
    B() {}
    ~B() {}
};

class C : virtual public B
{
public:
    C() {}
    ~C() {}
};

void main( void )
{
    B b;
    //C c;
    return;
}
```

注意：构造函数是继承实现的关键，每次子类对象构造时，首先调用的是父类的构造函数，然后才是自己的。

面试题 16：访问基类的私有虚函数

写出以下程序的输出结果：

```
#include <iostream.h>
class A
```

```

{
    virtual void g()
    {
        cout << "A::g" << endl;
    }
private:
    virtual void f()
    {
        cout << "A::f" << endl;
    }
};
class B : public A
{
    void g()
    {
        cout << "B::g" << endl;
    }
    virtual void h()
    {
        cout << "B::h" << endl;
    }
};
typedef void( *Fun )( void );
void main()
{
    B b;
    Fun pFun;
    for(int i = 0 ; i < 3; i++)
    {
        pFun = ( Fun )*( ( int* ) * ( int* )( &b ) + i );
        pFun();
    }
}

```

输出结果:

```

B::g
A::f
B::h

```

注意: 本题主要考察了面试者对虚函数的理解程度。一个对虚函数不了解的人很难正确的做出本题。在学习面向对象的多态性时一定要深刻理解虚函数表的工作原理。

面试题 17: 简述类成员函数的重写、重载和隐藏的区别

(1) 重写和重载主要有以下几点不同。

- ☐ 范围的区别: 被重写的和重写的函数在两个类中, 而重载和被重载的函数在同一个类中。
- ☐ 参数的区别: 被重写函数和重写函数的参数列表一定相同, 而被重载函数和重载函数的参数列表一定不同。
- ☐ virtual 的区别: 重写的基类中被重写的函数必须要有 virtual 修饰, 而重载函数和被重载函数可以被

virtual 修饰，也可以没有。

(2) 隐藏和重写、重载有以下几点不同。

- ❑ 与重载的范围不同：和重写一样，隐藏函数和被隐藏函数不在同一个类中。
- ❑ 参数的区别：隐藏函数和被隐藏的函数的参数列表可以相同，也可不同，但是函数名肯定要相同。当参数不相同时，无论基类中的参数是否被 virtual 修饰，基类的函数都是被隐藏，而不是被重写。

说明：虽然重载和覆盖都是实现多态的基础，但是两者实现的技术完全不相同，达到的目的也是完全不同的，覆盖是动态态绑定的多态，而重载是静态绑定的多态。

面试题 18：简述多态实现的原理

编译器发现一个类中有虚函数，便会立即为此类生成虚函数表 vtable。虚函数表的各表项为指向对应虚函数的指针。编译器还会在此类中隐含插入一个指针 vptr（对 vc 编译器来说，它插在类的第一个位置上）指向虚函数表。调用此类的构造函数时，在类的构造函数中，编译器会隐含执行 vptr 与 vtable 的关联代码，将 vptr 指向对应的 vtable，将类与此类的 vtable 联系了起来。另外在调用类的构造函数时，指向基础类的指针此时已经变成指向具体的类的 this 指针，这样依靠此 this 指针即可得到正确的 vtable，。如此才能真正与函数体进行连接，这就是动态联编，实现多态的基本原理。

注意：一定要区分虚函数，纯虚函数、虚拟继承的关系和区别。牢记虚函数实现原理，因为多态 C++ 面试的重要考点之一，而虚函数是实现多态的基础。

面试题 19：链表和数组有什么区别

数组和链表有以下几点不同：

(1) 存储形式：数组是一块连续的空间，声明时就要确定长度。链表是一块可不连续的动态空间，长度可变，每个结点要保存相邻结点指针。

(2) 数据查找：数组的线性查找速度快，查找操作直接使用偏移地址。链表需要按顺序检索结点，效率低。

(3) 数据插入或删除：链表可以快速插入和删除结点，而数组则可能需要大量数据移动。

(4) 越界问题：链表不存在越界问题，数组有越界问题。

说明：在选择数组或链表数据结构时，一定要根据实际需要进行选择。数组便于查询，链表便于插入删除。数组节省空间但是长度固定，链表虽然变长但是占了更多的存储空间。

面试题 20：怎样把一个单链表反序

(1) 反转一个链表。循环算法。

```
List reverse(List n)
{
    if(!n) //判断链表是否为空，为空即退出。
    {
        return n;
    }
    list cur = n.next; //保存头结点的下个结点
    list pre = n; //保存头结点
    list tmp;
```

```

pre.next = null;           //头结点的指针指空，转换后变尾结点
while ( NULL != cur.next ) //循环直到 cur.next 为空
{
    tmp = cur;              //实现如图 10.3—图 10.5 所示
    tmp.next = pre
    pre = tmp;

    cur = cur.next;
}
return tmp;                //f 返回头指针
}

```

(2) 反转一个链表。递归算法。

```

List *reverse( List *oldList, List *newHead = NULL )
{
    List *next = oldList-> next;    //记录上次翻转后的链表
    oldList-> next = newHead;        //将当前结点插入到翻转后链表的开头
    newHead = oldList;              //递归处理剩余的链表

    return ( next==NULL )? newHead: reverse( t, newHead );
}

```

说明：循环算法就是图 10.2—图 10.5 的移动过程，比较好理解和想到。递归算法的设计虽有一点难度，但是理解了循环算法，再设计递归算法就简单多了。

面试题 21：简述队列和栈的异同

队列和栈都是线性存储结构，但是两者的插入和删除数据的操作不同，队列是“先进先出”，栈是“后进先出”。

注意：区别栈区和堆区。堆区的存取是“顺序随意”，而栈区是“后进先出”。栈由编译器自动分配释放，存放函数的参数值，局部变量的值等。其操作方式类似于数据结构中的栈。堆一般由程序员分配释放，若程序员不释放，程序结束时可能由 OS 回收。分配方式类似于链表。

它与本题中的堆和栈是两回事。堆栈只是一种数据结构，而堆区和栈区是程序的不同内存存储区域。

面试题 22：能否用两个栈实现一个队列的功能

结点结构体：

```

typedef struct node
{
    int data;
    node *next;
}node,*LinkStack;

```

创建空栈：

```

LinkStack CreateNULLStack( LinkStack &S)
{
    S = (LinkStack)malloc( sizeof( node ) );    //申请新结点
    if( NULL == S)
    {
        printf("Fail to malloc a new node.\n");
    }
}

```

```

        return NULL;
    }

    S->data = 0; //初始化新结点
    S->next = NULL;

    return S;
}

```

栈的插入函数:

```

LinkStack Push( LinkStack &S, int data)
{
    if( NULL == S) //检验栈
    {
        printf("There no node in stack!");
        return NULL;
    }

    LinkStack p = NULL;
    p = (LinkStack)malloc( sizeof( node ) ); //申请新结点
    if( NULL == p)
    {
        printf("Fail to malloc a new node.\n");
        return S;
    }
    if( NULL == S->next)
    {
        p->next = NULL;
    }
    else
    {
        p->next = S->next;
    }
    p->data = data; //初始化新结点
    S->next = p; //插入新结点

    return S;
}

```

出栈函数:

```

node Pop( LinkStack &S)
{
    node temp;
    temp.data = 0;
    temp.next = NULL;

    if( NULL == S) //检验栈
    {
        printf("There no node in stack!");
        return temp;
    }
    temp = *S;
}

```

```

    if( S->next == NULL )
    {
        printf("The stack is NULL,can't pop!\n");
        return temp;
    }

    LinkStack p = S ->next;           //节点出栈
    S->next = S->next->next;
    temp = *p;

    free( p );
    p = NULL;

    return temp;
}

```

双栈实现队列的入队函数:

```

LinkStack StackToQueuePush( LinkStack &S, int data)
{
    node n;
    LinkStack S1 = NULL;
    CreateNULLStack( S1 );           //创建空栈

    while( NULL != S->next )         //S 出栈入 S1
    {
        n = Pop( S );
        Push( S1, n.data );
    }
    Push( S1, data );                //新结点入栈

    while( NULL != S1->next )        //S1 出栈入 S
    {
        n = Pop( S1 );
        Push( S, n.data );
    }

    return S;
}

```

说明：用两个栈能够实现一个队列的功能，那用两个队列能否实现一个队列的功能呢？结果是否定的，因为栈是先进后出，将两个栈连在一起，就是先进先出。而队列是现先进先出，无论多少个连在一起都是先进先出，而无法实现先进后出。

面试题 23：计算一颗二叉树的深度

深度的计算函数:

```

int depth(BiTree T)
{
    if(!T) return 0;                //判断当前结点是否为叶子结点
}

```

```

int d1= depth(T->lchild);           //求当前结点的左孩子树的深度
int d2= depth(T->rchild);           //求当前结点的右孩子树的深度
return (d1>d2?d1:d2)+1;
}

```

注意：根据二叉树的结构特点，很多算法都可以用递归算法来实现。

面试题 24：编码实现直接插入排序

直接插入排序编程实现如下：

```

#include<iostream.h>
void main( void )
{
    int ARRAY[10] = { 0, 6, 3, 2, 7, 5, 4, 9, 1, 8 };

    int i,j;
    for( i = 0; i < 10; i++)
    {
        cout<<ARRAY[i]<<" ";
    }

    cout<<endl;

    for( i = 2; i <= 10; i++ )           //将 ARRAY[2],...,ARRAY[n]依次按序插入
    {

        if(ARRAY[i] < ARRAY[i-1])         //如果 ARRAY[i]大于一切有序的数值,
                                           //ARRAY[i]将保持原位不动
        {
            ARRAY[0] = ARRAY[i];         //将 ARRAY[0]看做是哨兵,是 ARRAY[i]的副本
            j = i - 1;

            do{                             //从右向左在有序区 ARRAY[1. . i-1]中
                                           //查找 ARRAY[i]的插入位置
            ARRAY[j+1] = ARRAY[j];         //将数值大于 ARRAY[i]记录后移
            j-- ;
            }while( ARRAY[0] < ARRAY[j] );
            ARRAY[j+1]=ARRAY[0];           //ARRAY[i]插入到正确的位置上
        }
    }

    for( i = 0; i < 10; i++)
    {
        cout<<ARRAY[i]<<" ";
    }
    cout<<endl;
}

```

注意：所有为简化边界条件而引入的附加结点（元素）均可称为哨兵。引入哨兵后使得查找循环条件的时间大约减少了一半，对于记录数较大的文件节约的时间就相当可观。类似于排序这样使用频率非常高的算法，要尽可能地减少其运行时间。所以不能把上述算法中的哨兵视为雕虫小技。

面试题 25：编码实现冒泡排序

冒泡排序编程实现如下：

```
#include <stdio.h>
#define LEN 10                                //数组长度
void main( void )
{
    int ARRAY[10] = { 0, 6, 3, 2, 7, 5, 4, 9, 1, 8 }; //待排序数组
    printf( "\n" );
    for( int a = 0; a < LEN; a++ )                //打印数组内容
    {
        printf( "%d ", ARRAY[a] );
    }
    int i = 0;
    int j = 0;
    bool isChange;                                //设定交换标志
    for( i = 1; i < LEN; i++ )
    {
        //最多做 LEN-1 趟排序
        //本趟排序开始前,交换标志应为假
        //对当前无序区 ARRAY[i..LEN]自下向上扫描
        isChange = 0;
        for( j = LEN-1; j >= i; j-- )
        {
            if( ARRAY[j+1] < ARRAY[j] )
            {
                //交换记录
                //ARRAY[0]不是哨兵,仅做暂存单元
                ARRAY[0] = ARRAY[j+1];
                ARRAY[j+1] = ARRAY[j];
                ARRAY[j] = ARRAY[0];
                isChange = 1;                        //发生了交换,故将交换标志置为真
            }
        }
        printf( "\n" );
        for( a = 0; a < LEN; a++ )                //打印本次排序后数组内容
        {
            printf( "%d ", ARRAY[a] );
        }
        if( !isChange )                            //本趟排序未发生交换,提前终止算法
        {
            break;
        }
    }
    printf( "\n" );
    return;
}
```

面试题 26：编码实现直接选择排序

```
#include "stdio.h"
#define LEN 9
void main( void )
{
    int ARRAY[LEN]={ 5, 6, 8, 2, 4, 1, 9, 3, 7 };           //待序数组

    printf("Before sorted:\n");
    for( int m = 0; m < LEN; m++ )                         //打印排序前数组
    {
        printf( "%d  ", ARRAY[m] );
    }
    for (int i = 1; i <= LEN - 1; i++)                      //选择排序
    {
        int t = i - 1;
        int temp = 0;
        for (int j = i; j < LEN; j++)
        {
            if (ARRAY[j] < ARRAY[t])
            {
                t = j;
            }
        }
        if (t != (i - 1))
        {
            temp = ARRAY[i - 1];
            ARRAY[i - 1] = ARRAY[t];
            ARRAY[t] = temp;
        }
    }
    printf( "\n" );
    printf("After sorted:\n");
    for( i = 0; i < LEN; i++ )                             //打印排序后数组
    {
        printf( "%d  ", ARRAY[i] );
    }
    printf( "\n" );
}
```

注意：在直接选择排序中，具有相同关键码的对象可能会颠倒次序，因而直接选择排序算法是一种不稳定的排序方法。在本例中只是例举了简单的整形数组排序，肯定不会有什么问题。但是在复杂的数据元素序列组合中，只是根据单一的某一个关键值排序，直接选择排序则不保证其稳定性，这是直接选择排序的一个弱点。

面试题 27：编程实现堆排序

堆排序编程实现：

```
#include <stdio.h>
```

```

void createHeep(int ARRAY[],int sPoint, int Len)           //生成大根堆
{
    while( ( 2 * sPoint + 1 ) < Len )
    {
        int mPoint = 2 * sPoint + 1 ;
        if( ( 2 * sPoint + 2 ) < Len )
        {
            if(ARRAY[ 2 * sPoint + 1 ] < ARRAY[ 2 * sPoint + 2 ] )
            {
                mPoint = 2*sPoint+2;
            }
        }
        if(ARRAY[ sPoint ] < ARRAY[ mPoint ])             //堆被破坏，需要重新调整
        {
            int tmpData= ARRAY[ sPoint ];                //交换 sPoint 与 mPoint 的数据
            ARRAY[ sPoint ] = ARRAY[ mPoint ];
            ARRAY[ mPoint ] = tmpData;

            sPoint = mPoint ;
        }
        else
        {
            break;                                         //堆未破坏，不再需要调整
        }
    }
    return;
}

void heapSort( int ARRAY[], int Len )                     //堆排序
{
    int i=0;

    for ( i = ( Len / 2 - 1 ); i >= 0; i-- )               //将 Hr[0, Lenght-1]建成大根堆
    {
        createHeep(ARRAY, i, Len);
    }
    for ( i = Len - 1; i > 0; i-- )
    {
        int tmpData = ARRAY[0];                           //与最后一个记录交换
        ARRAY[0] = ARRAY[i];
        ARRAY[i] = tmpData;

        createHeep( ARRAY, 0, i );                         //将 H.r[0..i]重新调整为大根堆
    }

    return;
}

int main( void )

```

```

{
    int ARRAY[] = { 5, 4, 7, 3, 9, 1, 6, 8, 2 };

    printf("Before sorted:\n");
    for ( int i = 0; i < 9; i++ )
    {
        printf("%d ", ARRAY[i]);
    }
    printf("\n");

    heapSort( ARRAY, 9 );

    printf("After sorted:\n");
    for( i = 0; i < 9; i++ )
    {
        printf( "%d ", ARRAY[i] );
    }
    printf( "\n" );
    return 0;
}

```

说明：堆排序，虽然实现复杂，但是非常的实用。另外读者可是自己设计实现小堆排序的算法。虽然和大堆排序的实现过程相似，但是却可以加深对堆排序的记忆和理解。

面试题 28：编程实现基数排序

```

#include <stdio.h>
#include <malloc.h>
#define LEN 8
typedef struct node
{
    int data;
    struct node * next;
}node,*QueueNode;

typedef struct Queue
{
    QueueNode front;
    QueueNode rear;
}Queue,*QueueLink;

QueueLink CreateNullQueue( QueueLink &Q)
{
    Q = NULL;
    Q = ( QueueLink )malloc( sizeof( Queue ) );
    if( NULL == Q )
    {
        printf("Fail to malloc null queue!\n");
        return NULL;
    }
}

```

```

    Q->front = ( QueueNode )malloc( sizeof( node ) );
    Q->rear  = ( QueueNode )malloc( sizeof( node ) );
    if( NULL == Q->front || NULL == Q->rear )
    {
        printf("Fail to malloc a new queue's front or rear!\n");
        return NULL;
    }
    Q->rear = NULL;
    Q->front->next= Q->rear;

    return Q;
}
int lenData( node data[], int len)                //计算队列中各结点的数据的最大位数
{
    int m = 0;
    int temp = 0;
    int d;
    for( int i = 0; i < len; i++)
    {
        d = data[i].data;
        while( d > 0)
        {
            d /= 10;
            temp ++;
        }
        if( temp > m )
        {
            m = temp;
        }
        temp = 0;
    }
    return m;
}
QueueLink Push( QueueLink &Q , node node )      //将数据压入队列
{
    QueueNode p1,p;
    p =( QueueNode )malloc( sizeof( node ) );
    if( NULL == p )
    {
        printf("Fail to malloc a new node!\n");
        return NULL;
    }
    p1 = Q->front;
    while(p1->next != NULL)
    {
        p1 = p1->next;
    }
    p->data = node.data;
    p1->next = p;
    p->next = Q->rear;
}

```

```

        return NULL;
    }
    node Pop( QueueLink &Q)                //数据出队列
    {
        node temp;
        temp.data = 0;
        temp.next = NULL;
        QueueNode p;
        p = Q->front->next;
        if( p != Q->rear )
        {
            temp = *p;
            Q->front->next = p->next;
            free( p );
            p = NULL;
        }
        return temp;
    }
    int IsEmpty( QueueLink Q)
    {
        if( Q->front->next == Q->rear )
        {
            return 0;
        }
        return 1;
    }
    int main( void )
    {

        int i = 0;
        int Max = 0;                        //记录结点中数据的最大位数
        int d = 10;
        int power = 1;
        int k = 0;
        node Array[LEN] ={{450, NULL}, {32,NULL}, { 781,NULL}, { 57 ,NULL},组
                           { 145,NULL},{ 613,NULL},{ 401,NULL},{ 594,NULL}};
                           //队列结点数

        QueueLink Queue[10];
        for( i = 0; i < 10; i++)
        {
            CreateNullQueue( Queue[i]);    //初始化队列数组
        }
        for( i = 0; i < LEN; i++)
        {
            printf("%d  ",Array[i].data);
        }
        printf("\n");

        Max = lenData( Array, LEN );        //计算数组中关键字的最大位数
        printf("%d\n",Max);
    }

```

```

for(int j = 0; j < Max; j++)                //按位排序
{
    if(j == 0) power = 1;
    else power = power * d;

    for(i = 0; i < LEN; i++)
    {
        k = Array[i].data / power - (Array[i].data / (power * d)) * d;
        Push( Queue[k], Array[i] );
    }

    for(int l = 0, k = 0; l < d; l++)        //排序后出队列重入数组
    {
        while( IsEmpty( Queue[l] ) )
        {
            Array[k++] = Pop( Queue[l] );
        }
    }
    for( int t = 0; t < LEN; t++)
    {
        printf("%d  ", Array[t].data);
    }
    printf("\n");
}

return 0;
}

```

说明：队列为基数排序的实现提供了很大的方便，适当的数据机构可以减少算法的复杂度，让更多的算法实现更容易。

面试题 29：谈谈你对编程规范的理解或认识

编程规范可总结为：程序的可行性、可读性、可移植性以及可测试性。

说明：这是编程规范的总纲目，面试者不一定要去背诵上面给出的那几个例子，应该去理解这几个例子说明的问题，想一想，自己如何解决可行性、可读性、可移植性以及可测试性这几个问题，结合以上几个例子和自己平时的编程习惯来回答这个问题。

面试题 30：short i = 0; i = i + 1L; 这两句有错吗

代码一是错的，代码二是正确的。

说明：在数据安全的情况下大类型的数据向小类型的数据转换一定要显示的强制类型转换。

面试题 31：&&和&、||和|有什么区别

(1) &和|对操作数进行求值运算，&&和||只是判断逻辑关系。

(2) &&和||在判断左侧操作数就能确定结果的情况下就不再对右侧操作数求值。

注意：在编程的时候有些时候将&&或||替换成&或|没有出错，但是其逻辑是错误的，可能会导致不可预想的后果（比如当两个操作数一个是1另一个是2时）。

面试题 32：C++的引用和 C 语言的指针有什么区别

指针和引用主要有以下区别：

(1) 引用必须被初始化，但是不分配存储空间。指针不声明时初始化，在初始化的时候需要分配存储空间。

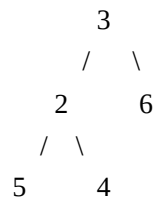
(2) 引用初始化以后不能被改变，指针可以改变所指的对象。

(3) 不存在指向空值的引用，但是存在指向空值的指针。

注意：引用作为函数参数时，会引发一定的问题，因为让引用作参数，目的就是想改变这个引用所指向地址的内容，而函数调用时传入的是实参，看不出函数的参数是正常变量，还是引用，因此可能会引发错误。所以使用时一定要小心谨慎。

面试题 33：在二元树中找出和为某一值的所有路径

输入一个整数和一棵二元树。从树的根结点开始往下访问，一直到叶结点所经过的所有结点形成一条路径。打印出和与输入整数相等的所有路径。例如，输入整数 9 和如下二元树：



则打印出两条路径：3，6 和 3，2，4。

【答案】

```
typedef struct path
{
    BiTNode* tree;           //结点数据成员
    struct path* next;       //结点指针成员
}PATH,*pPath;
```

初始化树的结点栈：

```
void init_path( pPath* L )
{
    *L = ( pPath )malloc( sizeof( PATH ) );    //创建空树
    (*L)->next = NULL;
}
```

树结点入栈函数：

```
void push_path(pPath H, pBTree T)
{
    pPath p = H->next;
    pPath q = H;

    while( NULL != p )
```

```

    {
        q = p;
        p = p->next;
    }

    p = ( pPath )malloc( sizeof( PATH ) );           //申请新结点
    p->next = NULL;                                   //初始化新结点
    p->tree = T;

    q->next = p;                                     //新结点入栈
}

```

树结点打印函数:

```

void print_path( pPath L )
{
    pPath p = L->next;

    while( NULL != p )                               //打印当前栈中所有数据
    {
        printf("%d, ", p->tree->data);
        p = p->next;
    }
}

```

树结点出栈函数:

```

void pop_path( pPath H )
{
    pPath p = H->next;
    pPath q = H;

    if( NULL == p )                                  //检验当前栈是否为空
    {
        printf("Stack is null!\n");
        return;
    }
    p = p->next;
    while( NULL != p )                               //出栈
    {
        q = q->next;
        p = p->next;
    }
    free( q->next );                                  //释放出栈结点空间
    q->next = NULL;
}

```

判断结点是否为叶子结点:

```

int IsLeaf(pBTree T)
{
    return ( T->lchild == NULL )&&( T->rchild==NULL );
}

```

查找符合条件的路径:

```

int find_path(pBTree T, int sum, pPath L)

```

```

{
    push_path( L, T);
    record += T->data;

    if( ( record == sum ) && ( !isLeaf( T ) ) )    //打印符合条件的当前路径
    {
        print_path( L );
        printf( "\n" );
    }

    if( T->lchild != NULL )    //递归查找当前节点的左孩子
    {
        find_path( T->lchild, sum, L);
    }
    if( T->rchild != NULL )    //递归查找当前节点的右孩子
    {
        find_path( T->rchild, sum, L);
    }

    record -= T->data;
    pop_path(L);
    return 0;
}

```

注意：数据结构一定要活学活用，例如本题，把所有的结点都压入栈，而不符合条件的结点弹出栈，很容易实现了有效路径的查找。虽然用链表也可以实现，但是用栈更利于理解这个问题，即适当的数据结构为更好的算法设计提供了有利的条件。

面试题 34：写一个“标准”宏 MIN

写一个“标准”宏 MIN，这个宏输入两个参数并且返回较小的一个。

【答案】

```
#define min(a,b)((a)<=(b)?(a):(b))
```

注意：在调用时一定要注意这个宏定义的副作用，如下调用：

```
((++*p)<=(x)?(++*p):(x))。
```

p 指针就自加了两次，违背了 MIN 的本意。

面试题 35：typedef 和 define 有什么区别

(1) 用法不同：typedef 用来定义一种数据类型的别名，增强程序的可读性。define 主要用来定义常量，以及书写复杂使用频繁的宏。

(2) 执行时间不同：typedef 是编译过程的一部分，有类型检查的功能。define 是宏定义，是预编译的部分，其发生在编译之前，只是简单的进行字符串的替换，不进行类型的检查。

(3) 作用域不同：typedef 有作用域限定。define 不受作用域约束，只要是在 define 声明后的引用都是正确的。

(4) 对指针的操作不同：typedef 和 define 定义的指针时有很大的区别。

注意：typedef 定义是语句，因为句尾要加上分号。而 define 不是语句，千万不能在句尾加分号。

面试题 36: 关键字 const 是什么

const 用来定义一个只读的变量或对象。主要优点: 便于类型检查、同宏定义一样可以方便地进行参数的修改和调整、节省空间, 避免不必要的内存分配、可为函数重载提供参考。

说明: const 修饰函数参数, 是一种编程规范的要求, 便于阅读, 一看即知这个参数不能被改变, 实现时不易出错。

面试题 37: static 有什么作用

static 在 C 中主要用于定义全局静态变量、定义局部静态变量、定义静态函数。在 C++ 中新增了两种作用: 定义静态数据成员、静态函数成员。

注意: 因为 static 定义的变量分配在静态区, 所以其定义的变量的默认值为 0, 普通变量的默认值为随机数, 在定义指针变量时要特别注意。

面试题 38: extern 有什么作用

extern 标识的变量或者函数声明其定义在别的文件中, 提示编译器遇到此变量和函数时在其它模块中寻找其定义。

面试题 39: 流操作符重载为什么返回引用

在程序中, 流操作符 >> 和 << 经常连续使用。因此这两个操作符的返回值应该是一个仍旧支持这两个操作符的流引用。其他的数据类型都无法做到这一点。

注意: 除了在赋值操作符和流操作符之外的其他的一些操作符中, 如 +、-、*、/ 等却千万不能返回引用。因为这四个操作符的对象都是右值, 因此, 它们必须构造一个对象作为返回值。

面试题 40: 简述指针常量与常量指针区别

指针常量是指定义了一个指针, 这个指针的值只能在定义时初始化, 其他地方不能改变。常量指针是指定义了一个指针, 这个指针指向一个只读的对象, 不能通过常量指针来改变这个对象的值。

指针常量强调的是指针的不可改变性, 而常量指针强调的是指针对其所指对象的不可改变性。

注意: 无论是指针常量还是常量指针, 其最大的用途就是作为函数的形式参数, 保证实参在被调用函数中的不可改变特性。

面试题 41: 数组名和指针的区别

请写出以下代码的打印结果:

```
#include <iostream.h>
#include <string.h>
void main(void)
{
    char str[13]="Hello world!";
```

```

char *pStr="Hello world!";

cout<<sizeof(str)<<endl;
cout<<sizeof(pStr)<<endl;
cout<<strlen(str)<<endl;
cout<<strlen(pStr)<<endl;

return;
}

```

【答案】

打印结果：

```

13
4
12
12

```

注意：一定要记得数组名并不是真正意义上的指针，它的内涵要比指针丰富的多。但是当数组名当做参数传递给函数后，其失去原来的含义，变作普通的指针。另外要注意 sizeof 不是函数，只是操作符。

面试题 42：如何避免“野指针”

“野指针”产生原因及解决办法如下：

(1) 指针变量声明时没有被初始化。解决办法：指针声明时初始化，可以是具体的地址值，也可让它指向 NULL。

(2) 指针 p 被 free 或者 delete 之后，没有置为 NULL。解决办法：指针指向的内存空间被释放后指针应该指向 NULL。

(3) 指针操作超越了变量的作用范围。解决办法：在变量的作用域结束前释放掉变量的地址空间并且让指针指向 NULL。

注意：“野指针”的解决方法也是编程规范的基本原则，平时使用指针时一定要避免产生“野指针”，在使用指针前一定要检验指针的合法性。

面试题 43：常引用有什么作用

常引用的引入主要是为了避免使用变量的引用时，在不知情的情况下改变变量的值。常引用主要用于定义一个普通变量的只读属性的别名、作为函数的传入形参，避免实参在调用函数中被意外的改变。

说明：很多情况下，需要用常引用做形参，被引用对象等效于常对象，不能在函数中改变实参的值，这样的好处是有较高的易读性和较小的出错率。

面试题 44：编码实现字符串转化为数字

编码实现函数 atoi()，设计一个程序，把一个字符串转化为一个整型数值。例如数字：“5486321”，转化成字符：5486321。

【答案】

```

int myAtoi(const char * str)
{

```

```

int num = 0; //保存转换后的数值
int isNegative = 0; //记录字符串中是否有负号
int n = 0;
char *p = str;
if(p == NULL) //判断指针的合法性
{
    return -1;
}
while(*p++ != '\0') //计算数字字符串度
{
    n++;
}
p = str;
if(p[0] == '-') //判断数组是否有负号
{
    isNegative = 1;
}

char temp = '0';
for(int i = 0 ; i < n; i++)
{
    char temp = *p++;
    if(temp > '9' || temp < '0') //滤除非数字字符
    {
        continue;
    }
    if(num != 0 || temp != '0') //滤除字符串开始的 0 字符
    {
        temp -= 0x30; //将数字字符转换为数值
        num += temp *int( pow(10 , n - 1 -i) );
    }
}
if(isNegative) //如果字符串中有负号，将数值取反
{
    return (0 - num);
}
else
{
    return num; //返回转换后的数值
}
}

```

注意：此段代码只是实现了十进制字符串到数字的转化，读者可以自己实现 2 进制，8 进制，10 进制，16 进制的转化。

面试题 45：简述 strcpy、sprintf 与 memcpy 的区别

三者主要有以下不同之处：

(1) 操作对象不同，strcpy 的两个操作对象均为字符串，sprintf 的操作源对象可以是多种数据类型，

目的操作对象是字符串，memcpy 的两个对象就是两个任意可操作的内存地址，并不限于何种数据类型。

(2) 执行效率不同，memcpy 最高，strcpy 次之，sprintf 的效率最低。

(3) 实现功能不同，strcpy 主要实现字符串变量间的拷贝，sprintf 主要实现其他数据类型格式到字符串的转化，memcpy 主要是内存块间的拷贝。

说明：strcpy、sprintf 与 memcpy 都可以实现拷贝的功能，但是针对的对象不同，根据实际需求，来选择合适的函数实现拷贝功能。

面试题 46：用 C 编写一个死循环程序

```
while(1)
{ }
```

说明：很多种途径都可实现同一种功能，但是不同的方法时间和空间占用度不同，特别是对于嵌入式软件，处理器速度比较慢，存储空间较小，所以时间和空间优势是选择各种方法的首要考虑条件。

面试题 47：编码实现某一变量某位清 0 或置 1

给定一个整型变量 a，写两段代码，第一个设置 a 的 bit 3，第二个清 a 的 bit 3，在以上两个操作中，要保持其他位不变。

【答案】

```
#define BIT3 (0x1 << 3)
```

```
static int a;
```

设置 a 的 bit 3:

```
void set_bit3( void )
```

```
{
    a |= BIT3;                //将 a 第 3 位置 1
}
```

清 a 的 bit 3

```
void set_bit3( void )
```

```
{
    a &= ~BIT3;               //将 a 第 3 位清零
}
```

说明：在置或清变量或寄存器的某一位时，一定要注意不要影响其他位。所以用加减法是很难实现的。

面试题 48：评论下面这个中断函数

中断是嵌入式系统中重要的组成部分，这导致了很多编译开发商提供一种扩展——让标准 C 支持中断。具体代表事实是，产生了一个新的关键字 __interrupt。下面的代码就使用了 __interrupt 关键字去定义一个中断服务子程序(ISR)，请评论以下这段代码。

```
__interrupt double compute_area (double radius)
{
    double area = PI * radius * radius;
    printf(" Area = %f", area);
    return area;
}
```

```
}
```

【答案】

这段中断服务程序主要有以下四个问题：

- (1) ISR 不能返回一个值。
- (2) ISR 不能传递参数。
- (3) 在 ISR 中做浮点运算是不明智的。
- (4) printf()经常有重入和性能上的问题。

注意：本题的第三个和第四个问题虽不是考察的重点，但是如果能提到这两点可给面试官留下一个好印象。

面试题 49：构造函数能否为虚函数

构造函数不能是虚函数。而且不能在构造函数中调用虚函数，因为那样实际执行的是父类的对应函数，因为自己还没有构造好。析构函数可以是虚函数，而且，在一个复杂类结构中，这往往是必须的。析构函数也可以是纯虚函数，但纯虚析构函数必须有定义体，因为析构函数的调用是在子类中隐含的。

说明：虚函数的动态绑定特性是实现重载的关键技术，动态绑定根据实际的调用情况查询相应类的虚函数表，调用相应的虚函数。

面试题 50：谈谈你对面向对象的认识

面向对象可以理解成对待每一个问题，都是首先要确定这个问题由几个部分组成，而每一个部分其实就是一个对象。然后再分别设计这些对象，最后得到整个程序。传统的程序设计多是基于功能的思想来进行考虑和设计的，而面向对象的程序设计则是基于对象的角度来考虑问题。这样做能够使得程序更加的简洁清晰。

说明：编程中接触最多的“面向对象编程技术”仅仅是面向对象技术中的一个组成部分。发挥面向对象技术的优势是一个综合的技术问题，不仅需要面向对象的分析，设计和编程技术，而且需要借助必要的建模和开发工具。

1. 内存分配方式及其区别

内存分配方式主要有以下几种：

- **静态内存分配**：在编译时确定内存的大小和位置。常见于全局变量、静态变量和常量。这些变量的生命周期是整个程序运行期间。
- **自动内存分配（栈内存分配）**：在函数调用时分配，函数结束时自动释放。适用于局部变量和函数参数。栈内存分配速度较快，但由于栈的大小有限，容易导致栈溢出。
- **动态内存分配**：在运行时通过函数如 `malloc`、`calloc`、`realloc` 和 `free` 在堆上分配和释放内存。动态内存分配允许在程序运行时根据需要调整内存使用，但可能导致内存泄漏和碎片化。
- **内存映射**：使用 `mmap` 函数将文件或设备映射到内存中，适用于大文件的访问或共享内存。映射区域可以被读取、写入或执行。

2. `const char *p` 和 `char * const p` 的区别

- `const char *p`：`p` 是一个指向 `const char` 的指针，意味着 `p` 所指向的数据不能被修改，但指针本身可以指向其他 `char` 类型的地址。
- `char * const p`：`p` 是一个 `const` 指针，意味着指针 `p` 不能被修改（即，不能指向其他地址），但指针指向的数据可以被修改。

3. `extern "C"` 的作用

在 C++ 中，`extern "C"` 用于告诉编译器以 C 语言的方式来处理指定的函数或变量，主要用于实现 C++ 程序调用用 C 编译器编译的函数。这样做的原因是 C++ 支持函数重载，而 C 语言不支持。使用 `extern "C"` 可以避免由于 C++ 的名称修饰（name mangling）导致的链接错误。

4. 逆序链表

```
void reverseList(Node*& head) {
    Node* prev = nullptr;
    Node* curr = head;
    Node* next = nullptr;

    while (curr != nullptr) {
        next = curr->next; // 保存下一个节点
        curr->next = prev; // 反转当前节点的指针
        prev = curr;      // 移动 prev 和 curr 指针
        curr = next;
    }
    head = prev; // 更新头节点
}
```

5. 程序输出和内存分布

程序1:

```
#include <iostream>
using namespace std;

class A {
```

```

public:
    virtual void aa(){}
private:
    char k[3];
};

class B: public A {
public:
    virtual void bb(){}
};

int main() {
    cout<<"A's size is "<<sizeof(A)<<endl;
    cout<<"B's size is "<<sizeof(B)<<endl;
    return 0;
}

```

执行结果:

```

A's size is 8
B's size is 8

```

内存分布:

- 类 A 包含一个虚函数表指针和一个 3 字节的 char 数组。通常情况下，虚函数表指针占用 4 字节（在 32 位系统上），加上 3 字节数组，整体对齐到 8 字节。
- 类 B 继承自 A，除了 A 的虚函数表指针和内存布局，B 还包含一个虚函数表指针，因此 sizeof(B) 和 sizeof(A) 相同。

程序2:

```

#include <iostream>
using namespace std;

class A {
public:
    virtual void aa(){}
    virtual void aa2(){}
private:
    char k[3];
};

class B {
public:
    virtual void bb(){}
    virtual void bb2(){}
};

class C: public A, public B {
public:
    virtual void aa(){} //重写了A的aa()
    virtual void cc(){}
};

```

```
int main() {
    cout<<"A's size is "<<sizeof(A)<<endl;
    cout<<"B's size is "<<sizeof(B)<<endl;
    cout<<"C's size is "<<sizeof(C)<<endl;
    return 0;
}
```

执行结果:

```
A's size is 12
B's size is 8
C's size is 16
```

内存分布:

- 类 **A** 包含两个虚函数表指针（每个 4 字节），以及 3 字节的 `char` 数组，通常对齐到 12 字节。
- 类 **B** 包含两个虚函数表指针，通常对齐到 8 字节。
- 类 **C** 继承自 **A** 和 **B**，因此需要存储两个虚函数表指针，以及 **A** 和 **B** 的内存布局，加上 `cc` 方法所需的内存，总共 16 字节。

6. 继承控制

为了确保子类不能修改基类的某些成员变量，可以将这些变量声明为 `protected` 或 `private`。同时，继承方式应该是 `protected` 或 `private`。`public` 继承方式会暴露基类的公共和保护成员变量。

7. 引用与指针的区别

- **引用**：必须在声明时初始化，不能改变引用的对象。引用通常用于函数参数传递和返回值，可以提高效率并避免复制。
- **指针**：可以在运行时改变指向的对象，可以为空。指针允许更多的灵活性，但需要显式地管理内存和处理空指针。

8. 引用作为函数返回值

格式:

```
int& foo() {
    static int value = 42;
    return value;
}
```

好处:

- 避免了值复制的开销，直接返回对象的引用。
- 允许在函数外部直接修改返回的对象。

规则:

- 返回局部变量的引用是不安全的，因为函数结束后局部变量会被销毁。
- 引用必须引用一个有效的对象。

9. `static` 和 `const` 关键字的作用

- `static`:
 - **在函数内**: 将变量的生命周期扩展到整个程序运行期间, 保持其值。
 - **在类内**: 声明类的静态成员, 所有对象共享。
 - **在文件内**: 限制符号的链接范围, 仅在当前文件有效。
- `const`:
 - **常量数据**: 确保变量的值在初始化后不能修改。
 - **常量指针**: 指针指向的数据不能被修改。
 - **常量成员函数**: 保证成员函数不会修改对象的状态。

10. C 和 C++ 中 `struct` 和 `class` 的区别

- `struct`:
 - 默认访问权限是 `public`。
 - 默认继承方式是 `public`。
- `class`:
 - 默认访问权限是 `private`。
 - 默认继承方式是 `private`。

11. 内联函数和宏的差别

- **内联函数**:
 - 由编译器处理, 避免了函数调用的开销。
 - 支持类型检查和调试。
- **宏**:
 - 由预处理器处理, 直接进行文本替换。
 - 不支持类型检查, 可能导致错误和调试困难。

12. 反向遍历 `vector` 的错误

```
vector<int> array;  
array.push_back(1);  
array.push_back(2);  
array.push_back(3);  
for (vector<int>::size_type i = array.size() - 1; i >= 0; --i) {  
    cout << array[i] << endl;  
}
```

错误: `i` 的类型是 `vector<int>::size_type`, 它是无符号类型。`i` 递减到 0 后, 再减去 1 会变成最大值, 从而导致无限循环。应该使用带符号整数类型, 例如 `int`。

13. 面向对象的三个基本特征

- **封装**：将数据和操作数据的方法封装在一个类中，隐藏内部实现细节，对外提供接口。
- **继承**：允许创建新类基于已有类，从而复用已有的代码和功能。
- **多态**：通过相同的接口调用不同的实现，允许在运行时决定使用哪个方法。

14. 32 位 ARM 系统下结构体 `sizeof(A)`

```
struct A {  
    char t:4;  
    char k:4;  
    unsigned short i:8;  
    unsigned long m;  
};
```

- `char` 位域占用 1 字节 (8 位)，`unsigned short` 位域占用 1 字节 (8 位)，`unsigned long` 通常占用 4 字节 (32 位)。
- 对齐要求：`unsigned long` 需要对齐到 4 字节。

`sizeof(A)` = 12 字节 (8 字节用于位域 + 4 字节对齐)。

15. 程序的错误或效果

```
#define MAX 255  
int main() {  
    unsigned char A[MAX], i;  
    for (i = 0; i <= MAX; i++)  
        A[i] = i;  
}
```

错误：`i <= MAX` 的循环条件导致 `i` 超出 `unsigned char` 的范围，可能引发数组越界访问。`MAX` 为 255，但 `i` 从 0 到 255，总共 256 次迭代，会越界访问数组 `A` 的第 256 个元素。

16. 程序的错误或效果

```
void GetMemory(char **p, int num) {  
    *p = (char *)malloc(num);  
}  
void Test(void) {  
    char *str = NULL;  
    GetMemory(&str, 100);  
    strcpy(str, "hello");  
    printf(str);  
}
```

错误：

- `malloc` 的返回值未进行检查，可能会导致空指针解引用。
- `printf` 中使用了未格式化的字符串，应该使用 `printf("%s", str);`。

17. sizeof 计算

```
char str[] = "raysharp.cn";
char *p = str;
int n = 10;
```

- `sizeof(str)`: `str` 是一个数组，大小是字符串长度加上终止字符。"raysharp.cn" 的长度是 11 字节，所以 `sizeof(str) = 12`。
- `sizeof(p)`: `p` 是一个指针，在 32 位系统下，`sizeof(p) = 4` 字节。
- `sizeof(n)`: `n` 是一个 `int`，在 32 位系统下，`sizeof(n) = 4` 字节。

```
void Foo(char str[100]) {
    // 这里 str 是一个指针，不是数组
}
```

- `sizeof(str)`: 在函数参数中，`str` 被视为指针，`sizeof(str) = 4` 字节。

```
void *p = malloc(100);
```

- `sizeof(p)`: `p` 是一个指针，`sizeof(p) = 4` 字节。

18. 复制视频文件的函数

```
#include <fstream>

void copyFile(const std::string& sourcePath, const std::string& destPath) {
    std::ifstream src(sourcePath, std::ios::binary);
    std::ofstream dest(destPath, std::ios::binary);

    dest << src.rdbuf();
}
```

19. 交叉编译器的作用

交叉编译器用于在一种平台上编译代码，以便在另一种平台上运行。例如，在 Windows 上编译代码以在嵌入式 ARM 设备上运行。它允许开发者为目标平台生成可执行代码，而无需在目标平台上进行开发和测试。

20. Shell 脚本判断程序

```
#!/bin/bash
a=0
if [ $a -eq 0 ]; then
    echo "correct"
else
    echo "incorrect"
fi
```

21. 查找并删除 `.o` 文件

```
find . -type f -name "*.o" -delete
```

22. 判断表达式正确性

```
int a = 4;
(A) a += (a++); // a = 4 + 4; a = 8
(B) a += (++a); // a = 8 + 6; a = 14
(C) (a++) += a; // 错误, 不能将右值赋值给左值
(D) (++a) += (a++); // a = 5; a = 5 + 6; a = 11
```

23. 字符设备与块设备的区别

- **字符设备**: 数据按字符流的方式传输。没有固定的大小和结构。例子: 串口设备、终端。
- **块设备**: 数据以块的形式进行传输, 每个块具有固定的大小。适用于需要随机访问的存储设备。例子: 硬盘、USB 驱动器。

好的, 我们来逐一完成这些试题:

1. 不能做 `switch()` 的参数类型是?

`switch()` 语句的参数类型必须是整型数据 (`int`、`char`、`enum` 等), 不能是浮点型 (`float`、`double`)、指针或其他类型。具体来说, 以下类型不能用作 `switch()` 的参数:

- `float`
- `double`
- `char*` (指针类型)
- `void*` (空指针类型)

2. C++ 中如何声明 `const void f(void)` 函数为 C 程序中的库函数?

要在 C++ 中声明一个函数为 C 语言中的库函数, 需要使用 `extern "C"` 来防止 C++ 编译器对函数名进行名称修饰。声明 `const void f(void)` 为 C 语言中的库函数可以如下实现:

```
extern "C" void f(void);
```

3. `const char *p` 和 `char * const p` 的区别

- `const char *p`: `p` 是一个指向 `const char` 的指针, 意味着 `p` 所指向的数据不能被修改, 但指针本身可以指向其他 `char` 类型的地址。
- `char * const p`: `p` 是一个 `const` 指针, 意味着指针 `p` 不能改变它指向的地址, 但指针指向的数据可以被修改。

4. 面向对象程序设计的优点

- **封装**：将数据和操作数据的方法封装在一个类中，隐藏内部实现细节，对外提供接口。
- **继承**：允许创建新类基于已有类，从而复用已有的代码和功能。
- **多态**：通过相同的接口调用不同的实现，允许在运行时决定使用哪个方法。

5. 举例结构体中的一个 `char` 变量怎么拆分成两个变量

假设结构体中有一个 `char` 变量 `c`，我们可以将其拆分成两个 `char` 变量，如下：

```
struct Example {
    char c;
};

void splitChar(Example* e, char* high, char* low) {
    *high = (e->c >> 4) & 0x0F; // 高四位
    *low = e->c & 0x0F; // 低四位
}
```

6. BOOL, int, float, 指针类型的变量 `a` 与“零”的比较语句

- **BOOL**:

```
BOOL a;
if (a == 0) { /* ... */ }
```

- **int**:

```
int a;
if (a == 0) { /* ... */ }
```

- **float**:

```
float a;
if (a == 0.0f) { /* ... */ }
```

- **指针**:

```
void *a;
if (a == NULL) { /* ... */ }
```

7. 为 `Type *s` 在堆中分配空间

```
Type *s = (Type *)malloc(sizeof(Type));
```

8. 什么是动态特性？

动态特性通常指程序在运行时的行为特征。例如，在面向对象编程中，动态特性包括多态、动态绑定等，这允许在运行时根据实际对象类型来决定方法的调用。

9. 全局变量可不可以定义在可被多个 `.c` 文件包含的头文件中？为什么？

不建议将全局变量定义在可被多个 `.c` 文件包含的头文件中。因为这样会导致每个包含该头文件的 `.c` 文件都有一个全局变量的副本，从而引发链接错误。全局变量应在一个 `.c` 文件中定义，在头文件中只声明（`extern`）。

10. 已知二叉树后序遍历序列是 `dabec`，中序遍历序列是 `debac`，它的前序遍历序列是？

根据给定的后序和中序遍历序列，我们可以推断前序遍历序列。构造树的过程如下：

- 后序遍历的最后一个字符是树的根：`c`。
- 在中序遍历中，`c` 的左子树是 `debac` 中的 `deba`，右子树为空。
- 对 `deba` 进行相同的分析。

最终的前序遍历序列是：`c + dab + e`，即 `cdeb`。

11. 进程间通信的方式有哪些？

- **管道** (Pipes)：用于同一进程组或父子进程之间的通信。
- **命名管道** (Named Pipes)：用于不同进程之间的通信。
- **消息队列** (Message Queues)：允许进程之间以消息的形式交换数据。
- **共享内存** (Shared Memory)：多个进程可以访问同一块内存区域。
- **信号量** (Semaphores)：用于进程间的同步。
- **套接字** (Sockets)：可以用于不同计算机间的通信，支持网络通信。

12. 输出结果

```
#include <stdio.h>
union {
    int i;
    int j;
} a;

void main() {
    a.i = 5;
    a.j = 6;
    printf("%d", a.i);
}
```

输出结果是 `6`。由于 `union` 中的所有成员共享同一块内存，所以最后对 `a.j` 的赋值会覆盖 `a.i` 的值。

13. 实现数组的降序排序

```
#include <stdio.h>

void sort(int arr[], int size) {
    int temp;
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (arr[i] < arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

int main() {
    int array[] = {45, 56, 76, 234, 1, 34, 23, 2, 3};
    int size = sizeof(array) / sizeof(array[0]);
    sort(array, size);

    for (int i = 0; i < size; i++) {
        printf("%d ", array[i]);
    }
    return 0;
}
```

14. 程序问题

```
void GetMemory(char **p, int num) {
    *p = (char *)malloc(num);
}

void Test(void) {
    char *str = NULL;
    GetMemory(&str, 100);
    strcpy(str, "hello");
    printf("%s", str); // 错误：未格式化字符串
}
```

问题： `printf` 中应使用格式化字符串 `"%s"`。否则，程序可能会出现未定义行为。

15. 计算 `sizeof` 的值

```
char str[] = "raysharp.cn";
char *p = str;
int n = 10;
```

- `sizeof(str)`： `str` 是一个字符数组，包含 11 个字符（包括终止字符）。所以 `sizeof(str) = 12`。
- `sizeof(p)`： `p` 是一个 `char*` 指针，在 32 位系统下， `sizeof(p) = 4`。

- `sizeof(n)`: `n` 是一个 `int`, 在 32 位系统下, `sizeof(n) = 4`。

```
void Foo(char str[100]) {  
    // 参数 str 被视为 char* 类型  
    sizeof(str) // 实际上是 sizeof(char*), 即 4  
}
```

- `sizeof(str)`: 在函数参数中, `str` 被视为 `char*`, 所以 `sizeof(str) = 4`。

```
void *p = malloc(100);
```

- `sizeof(p)`: `p` 是 `void*` 指针, 在 32 位系统下, `sizeof(p) = 4`。

16. 32-bit 数 0x12345678 在 Little-endian 和 Big-endian 模式下的存放方式

- **Little-endian**: 低位字节存储在低地址, 存储顺序为: 78 56 34 12 从地址 0x4000 开始。
- **Big-endian**: 高位字节存储在低地址, 存储顺序为: 12 34 56 78 从地址 0x4000 开始。

17. 计算 long 变量中有多少个 bit 的值是 1

```
#include <stdio.h>  
  
int countBits(long num) {  
    int count = 0;  
    while (num) {  
        count += num & 1;  
        num >>= 1;  
    }  
    return count;  
}  
  
int main() {  
    long num = 0x12345678;  
    printf("Number of 1 bits: %d\n", countBits(num));  
    return 0;  
}
```

18. Linux 内核同步方法

- 互斥锁 (Mutexes)
- 自旋锁 (Spinlocks)
- 读写锁 (Read-Write Locks)
- 信号量 (Semaphores)
- 条件变量 (Condition Variables)

19. 用 shell 脚本判断变量 `a` 的值

```
#!/bin/bash

a=0
if [ $a -eq 0 ]; then
    echo "correct"
elif [ $a -eq 1 ]; then
    echo "incorrect"
else
    echo "Invalid value"
fi
```

20. 用 Linux 命令查找当前文件下所有 C 文件

```
find . -name "*.c"
```

22. 判断 A B C D 四个表达式的是否正确，并写出经过表达式中 `a` 的值

```
int a = 4;
```

(A) `a += (a++);`
// `a++` 先使用 `a` 的值，再增加 1。结果为 $4 + 4 = 8$ 。`a` 变成 5。

(B) `a += (++a);`
// `++a` 先增加 1，再使用 `a` 的值。结果为 $5 + 6 = 11$ 。`a` 变成 6。

(C) `(a++) += a;`
// `a++` 先使用 `a` 的值，再增加 1。结果为 $4 + 5 = 9$ 。`a` 变成 5。

(D) `(++a) += (a++);`
// `++a` 先增加 1，再使用 `a` 的值。结果为 $6 + 6 = 12$ 。`a` 变成 7。

希望这些答案对你有帮助！如果有任何问题或需要进一步的解释，请告诉我。

STM32

1. STM32的时钟系统如何配置？

- **答案：**STM32的时钟系统由多个时钟源组成，包括外部晶振（HSE）、内部振荡器（HSI）和PLL（相位锁环）。配置时钟系统需要设置时钟源、分频器和PLL参数。例如，可以通过STM32的时钟配置工具（如STM32CubeMX）进行配置，或者在代码中配置RCC寄存器，设置系统时钟、AHB时钟、APB时钟等参数。重要的是要确保系统时钟源稳定，并根据实际需求配置合适的频率。

2. 如何在STM32中实现定时器的PWM输出？

- **答案：**在STM32中实现PWM输出通常包括以下步骤：
 1. 配置定时器的时基单元（如TIM1、TIM2等），设置PWM的频率和周期。
 2. 配置PWM模式的输出通道，设置占空比。
 3. 启动定时器，并使能PWM输出通道。
 4. 根据需要，使用PWM信号控制外设或实现调光等功能。
- 代码示例：

```
// 配置定时器TIM2为PWM模式
TIM_HandleTypeDef htim2;
TIM_OC_InitTypeDef sConfigOC = {0};

// 初始化TIM2
htim2.Instance = TIM2;
htim2.Init.Prescaler = 0;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 1000 - 1; // PWM周期
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
HAL_TIM_Base_Init(&htim2);

// 配置PWM模式
sConfigOC.OCMode = TIM_OCMODE_PWM1;
sConfigOC.Pulse = 500; // 占空比50%
sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCNPolarity = TIM_OCNPOLARITY_LOW;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1);

// 启动PWM
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

3. 描述STM32的ADC转换流程，包括配置和读取过程。

- **答案：**STM32的ADC转换流程包括以下步骤：
 1. 配置ADC时钟和前置分频器，以确定ADC的转换速率。
 2. 配置ADC的输入通道、采样时间和转换模式（单次或连续）。
 3. 启动ADC转换，可以选择使用软件触发或硬件触发。
 4. 等待转换完成，通过ADC寄存器读取转换结果。
 5. 处理转换结果并根据需要执行后续操作（如数据处理或存储）。
- 代码示例：

```

ADC_HandleTypeDef hadc1;

// 初始化ADC1
hadc1.Instance = ADC1;
hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
hadc1.Init.ContinuousConvMode = DISABLE;
hadc1.Init.DiscontinuousConvMode = DISABLE;
hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
hadc1.Init.NbrOfConversion = 1;
HAL_ADC_Init(&hadc1);

// 配置ADC通道
ADC_ChannelConfTypeDef sConfig = {0};
sConfig.Channel = ADC_CHANNEL_0;
sConfig.Rank = ADC_REGULAR_RANK_1;
sConfig.SamplingTime = ADC_SAMPLETIME_3CYCLES;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);

// 启动转换
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
uint32_t adcValue = HAL_ADC_GetValue(&hadc1);

```

单片机 (MCU)

4. 单片机的中断系统如何工作？

- **答案：**单片机的中断系统通过中断请求（IRQ）信号响应外部或内部事件。中断系统包括中断源、中断向量表、中断控制器和中断服务程序（ISR）。当中断发生时，中断控制器会将中断请求传递给CPU，CPU中断当前执行的程序，跳转到对应的ISR处理函数。处理完中断后，CPU会恢复之前的程序执行。
- 关键点包括中断优先级设置、屏蔽中断和嵌套中断。

5. 如何在单片机中实现串口通信？

- **答案：**串口通信通常包括以下步骤：
 1. 配置串口波特率、数据位、停止位和校验位。
 2. 配置串口的发送和接收功能。
 3. 使用中断或轮询方式处理接收到的数据，并发送数据。
- 代码示例（基于8051单片机）：

```

// 初始化串口
void UART_Init() {
    TMOD = 0x20; // 定时器1，自动重装模式
    TH1 = 0xFD; // 波特率9600
    TL1 = 0xFD;
    TR1 = 1; // 启动定时器1
    SCON = 0x50; // 配置串口为8位数据位，1位停止位
    EA = 1; // 启用全局中断
    ES = 1; // 启用串口中断
}

```

```
// 串口发送数据
void UART_Send(char data) {
    SBUF = data;
    while (!TI); // 等待发送完成
    TI = 0; // 清除发送中断标志
}

// 串口接收中断服务函数
void UART_ISR() interrupt 4 {
    char received = SBUF; // 读取接收到的数据
    // 处理接收到的数据
}
```

Linux

6. 如何在Linux中使用内核模块来实现GPIO控制？

- 答案：编写一个Linux内核模块来控制GPIO涉及以下步骤：

1. 使用 `gpio.h` 头文件中的GPIO API来控制GPIO引脚。
2. 在模块初始化函数中配置GPIO引脚的方向和初始状态。
3. 在模块退出函数中清理GPIO配置。

- 代码示例：

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/gpio.h>
#include <linux/init.h>

static int gpio_pin = 23; // 假设我们使用GPIO 23

static int __init gpio_module_init(void) {
    int ret;

    ret = gpio_request(gpio_pin, "gpio_test");
    if (ret) {
        printk(KERN_ERR "Failed to request GPIO %d\n", gpio_pin);
        return ret;
    }

    gpio_direction_output(gpio_pin, 1); // 设置GPIO为输出并置为高电平
    printk(KERN_INFO "GPIO module initialized\n");

    return 0;
}

static void __exit gpio_module_exit(void) {
    gpio_set_value(gpio_pin, 0); // 置为低电平
    gpio_free(gpio_pin);
    printk(KERN_INFO "GPIO module exited\n");
}

module_init(gpio_module_init);
module_exit(gpio_module_exit);
```

```
MODULE_LICENSE("GPL");
```

7. 如何在Linux中实现一个简单的字符设备驱动？

- **答案：**实现字符设备驱动包括以下步骤：
 1. 定义设备文件操作结构体，包括打开、读取、写入等函数。
 2. 注册字符设备，分配设备号。
 3. 实现文件操作函数并在模块中注册。
 4. 在模块初始化和退出函数中处理设备的注册和注销。
- 代码示例：

```
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "my_char_device"

static int device_open(struct inode *inode, struct file *file) {
    return 0;
}

static ssize_t device_read(struct file *file, char __user *buffer,
size_t length, loff_t *offset) {
    const char *data = "Hello from kernel space!";
    size_t datalen = strlen(data);

    if (*offset >= datalen) return 0;
    if (length > datalen - *offset) length = datalen - *offset;

    if (copy_to_user(buffer, data + *offset, length)) return -EFAULT;

    *offset += length;
    return length;
}

static struct file_operations fops = {
    .open = device_open,
    .read = device_read,
};

static int __init char_device_init(void) {
    int result;
    result = register_chrdev(0, DEVICE_NAME, &fops);
    if (result < 0) {
        printk(KERN_ALERT "Failed to register device\n");
        return result;
    }
    printk(KERN_INFO "Device registered with major number %d\n",
result);
    return 0;
}

static void __exit char_device_exit(void) {
    unregister_chrdev(0, DEVICE_NAME);
```

```

    printk(KERN_INFO "Device unregistered\n");
}

module_init(char_device_init);
module_exit(char_device_exit);
MODULE_LICENSE("GPL");

```

RTOS

8. RTOS中任务间通信的主要方式有哪些？

- **答案：**RTOS中任务间通信的主要方式包括：
 1. **消息队列：**用于任务之间传递消息或数据，支持FIFO（先进先出）队列。
 2. **信号量：**用于同步任务或保护共享资源，支持二值信号量和计数信号量。
 3. **事件标志组：**用于多个任务之间的事件通知和同步。
 4. **共享内存：**多个任务可以通过共享内存区域直接读写数据。
 5. **管道：**用于任务之间的数据流传输。
- 每种通信机制都有其适用的场景和特性，选择合适的机制可以提高系统的效率和稳定性。

9. 在FreeRTOS中如何实现任务优先级的管理？

- **答案：**FreeRTOS通过任务优先级来管理任务调度。每个任务在创建时指定一个优先级，RTOS内核根据任务的优先级决定调度顺序。任务优先级的管理包括：
 1. **创建任务时指定优先级：**通过 `xTaskCreate` 函数设置任务的优先级。
 2. **修改任务优先级：**使用 `vTaskPrioritySet` 函数可以动态调整任务的优先级。
 3. **了解优先级调度：**高优先级任务会优先于低优先级任务执行。当高优先级任务变为就绪状态时，它会抢占低优先级任务的CPU时间。
- 代码示例：

```

TaskHandle_t xTaskHandle;

void vTaskFunction(void *pvParameters) {
    while (1) {
        // 任务执行代码
    }
}

void create_tasks() {
    xTaskCreate(vTaskFunction, "Task1", 1000, NULL, 1, &xTaskHandle); // 优先级1
    xTaskCreate(vTaskFunction, "Task2", 1000, NULL, 2, NULL); // 优先级2
}

void change_task_priority() {
    vTaskPrioritySet(xTaskHandle, 3); // 将任务优先级修改为3
}

```

C语言

10. C语言中的指针与数组有什么关系？

- **答案：**在C语言中，指针和数组有着密切的关系。数组名可以被看作是指向数组首元素的指针。指针可以通过加法和减法操作来访问数组的不同元素。数组名本身不能被改变（即不能指向其他地方），而指针可以重新赋值。
- 示例：

```
int arr[5] = {1, 2, 3, 4, 5};
int *ptr = arr; // 等同于 int *ptr = &arr[0];
printf("%d\n", ptr[2]); // 输出3，相当于 *(ptr + 2)
```

11. C语言中如何处理内存泄漏？

- **答案：**内存泄漏发生在程序分配了内存但未能释放或丢失对已分配内存的引用。处理内存泄漏的方法包括：
 1. **正确使用内存分配和释放函数：**如 `malloc`、`calloc`、`realloc` 和 `free`。
 2. **使用工具进行内存检测：**如 Valgrind、AddressSanitizer 等。
 3. **保持良好的编码习惯：**如确保每个 `malloc` 有对应的 `free`，避免忘记释放内存。
 4. **分析代码：**检查代码路径是否确保释放每个分配的内存块。

12. C语言中的 `volatile` 关键字有什么作用？

- **答案：**`volatile` 关键字用于修饰变量，表示该变量的值可能会被外部因素（如硬件或中断）改变，因此编译器在优化时不能对其进行缓存。它告诉编译器每次访问变量时都需要从内存中读取，而不是使用寄存器中的值。
- 示例：

```
volatile int flag;
void interrupt_handler() {
    flag = 1; // 外部中断可能会改变flag的值
}
void main() {
    while (!flag) {
        // 等待中断设置flag
    }
}
```

高级综合题目

13. 如何在RTOS下使用互斥量实现对共享资源的保护？

- **答案：**在RTOS中，互斥量（Mutex）用于保护共享资源，避免多个任务同时访问导致的数据冲突。实现步骤如下：
 1. **创建互斥量：**使用RTOS提供的API创建一个互斥量。
 2. **获取互斥量：**在访问共享资源前，任务必须获取互斥量。
 3. **释放互斥量：**在操作完成后，任务释放互斥量，以允许其他任务访问。
- 代码示例（FreeRTOS）：

```
SemaphoreHandle_t xMutex;
```

```

void task1(void *pvParameters) {
    if (xSemaphoreTake(xMutex, portMAX_DELAY)) {
        // 访问共享资源
        xSemaphoreGive(xMutex);
    }
}

void task2(void *pvParameters) {
    if (xSemaphoreTake(xMutex, portMAX_DELAY)) {
        // 访问共享资源
        xSemaphoreGive(xMutex);
    }
}

void create_tasks() {
    xMutex = xSemaphoreCreateMutex();
    xTaskCreate(task1, "Task1", 1000, NULL, 1, NULL);
    xTaskCreate(task2, "Task2", 1000, NULL, 1, NULL);
}

```

14. 如何在Linux内核中处理设备驱动程序的中断？

- 答案：在Linux内核中处理设备驱动程序的中断通常包括以下步骤：
 1. **注册中断处理程序**：使用 `request_irq` 函数注册中断处理程序，并指定中断号和处理函数。
 2. **中断处理函数**：编写中断处理函数来处理具体的中断逻辑，并可能在处理完成后清除中断标志。
 3. **释放中断**：在驱动卸载时，使用 `free_irq` 函数释放中断。
- 代码示例：

```

#include <linux/module.h>
#include <linux/interrupt.h>

static irqreturn_t irq_handler(int irq, void *dev_id) {
    printk(KERN_INFO "Interrupt handled\n");
    return IRQ_HANDLED;
}

static int __init irq_module_init(void) {
    int result;
    result = request_irq(IRQ_NUM, irq_handler, IRQF_SHARED, "my_irq",
        NULL);
    if (result) {
        printk(KERN_ERR "Failed to request IRQ\n");
        return result;
    }
    printk(KERN_INFO "IRQ module initialized\n");
    return 0;
}

static void __exit irq_module_exit(void) {
    free_irq(IRQ_NUM, NULL);
    printk(KERN_INFO "IRQ module exited\n");
}

```

```
}

module_init(irq_module_init);
module_exit(irq_module_exit);
MODULE_LICENSE("GPL");
```

15. 在STM32中，如何在中断服务例程中安全地使用全局变量？

- **答案：**在STM32中使用全局变量时，如果这些变量在中断服务例程（ISR）和主程序中都被访问，则需要确保数据的一致性。解决方法包括：

1. **使用 volatile 关键字：**告诉编译器不要优化该变量，确保每次都从内存中读取。
2. **避免在ISR中执行复杂计算：**尽量将复杂计算或数据处理放到任务中，ISR中仅设置标志或更新简单的数据。
3. **保护临界区：**使用互斥量或禁用中断来保护对全局变量的访问，防止数据竞态。

- 示例：

```
volatile int shared_variable;

void TIM2_IRQHandler(void) {
    shared_variable++;
}

void main(void) {
    while (1) {
        if (shared_variable > 100) {
            // 处理逻辑
        }
    }
}
```

这些问题涵盖了嵌入式系统中的各种技术挑战，可以帮助你更好地准备面试。

如何设计一个高效的实时调度系统？

答案：

设计高效的实时调度系统需要考虑以下几个方面：

- **任务优先级**：将任务分配给不同的优先级，以确保高优先级任务可以抢占低优先级任务。常见的算法包括优先级抢占、静态优先级和动态优先级。
- **调度算法**：常见的调度算法有循环调度（Round-Robin）、最早截止时间优先（EDF）和固定优先级调度（Fixed Priority Scheduling）。选择适合应用场景的算法，并确保任务在其截止时间前完成。
- **时间片管理**：设定适当的时间片长度，避免过长导致响应时间不够及时，或过短导致上下文切换频繁。
- **任务模型**：对每个任务的执行时间和周期性进行建模，并进行性能评估以优化调度策略。
- **实时性保证**：利用分析工具对系统进行实时性分析，确保最坏情况下的响应时间和任务完成时间满足要求。

如何处理嵌入式系统中的内存碎片问题？

答案：

内存碎片问题可以通过以下方法处理：

- **内存池**：预分配固定大小的内存块，并在任务运行时分配这些块。这样可以减少内存碎片的发生，并且简化内存管理。
- **伙伴系统**：将内存划分为不同大小的块，通过合并和分割操作来减少外部碎片。伙伴系统在内存分配和释放时动态调整块的大小。
- **内存压缩**：在空闲内存区域之间填充数据，减少内存碎片。也可以在应用层使用自定义的分配策略。
- **垃圾回收**：在需要的时候定期进行垃圾回收，清理不再使用的内存，虽然这在实时系统中可能会引入不可预测的延迟。

描述如何在没有操作系统的情况下实现任务切换。

答案：

- **硬件定时器**：使用硬件定时器（如定时中断）来触发任务切换。定时器定期中断CPU，并调用任务调度程序。
- **上下文保存**：在任务切换时，需要保存当前任务的上下文（寄存器状态、程序计数器等）到任务控制块（TCB）中。
- **上下文恢复**：从下一个任务的TCB中恢复上下文，恢复到该任务的执行状态。
- **任务栈管理**：每个任务有自己的栈空间，任务切换时需要保存和恢复任务的栈指针。
- **调度算法**：实现一个简单的调度算法（如轮询）来决定下一个要运行的任务。

在嵌入式系统中，如何实现一个高效的互斥机制？

答案：

- **自旋锁**：在锁竞争时间短的情况下，使用自旋锁避免上下文切换。自旋锁在锁被占用时不断轮询，直到锁被释放。

- **原子操作**：使用原子操作（如Test-and-Set）来实现简单的互斥。原子操作能确保对共享变量的操作是不可分割的。
- **互斥量**：使用互斥量（Mutex）来管理对共享资源的访问。互斥量能够提供更强的互斥保证，并适用于较长时间的锁定。
- **优先级继承**：为避免优先级反转问题，可以实现优先级继承协议，即低优先级任务在持有共享资源时，临时提升到高优先级。

解释如何在嵌入式系统中进行时间片轮转调度。

答案：

- **时间片设置**：将CPU时间划分为固定长度的时间片（时间量程），并为每个任务分配一个时间片。
- **任务队列**：维护一个任务队列，任务按照进入队列的顺序安排执行。每个任务获得一个时间片来执行其代码。
- **任务切换**：当一个任务的时间片用完时，调度程序会将任务放到队列末尾，然后选择队列中的下一个任务来运行。
- **上下文切换**：保存当前任务的上下文状态（寄存器、程序计数器等），恢复下一个任务的上下文状态。
- **轮转机制**：继续循环队列中的任务，确保所有任务都能公平地获得CPU时间。

如何优化嵌入式系统的启动时间？

答案：

- **代码优化**：精简启动代码，只进行必要的初始化。减少不必要的启动步骤和复杂的初始化过程。
- **延迟初始化**：将某些不关键的初始化操作推迟到系统运行后进行。比如，延迟初始化不关键的外设，等系统稳定后再进行配置。
- **并行初始化**：如果硬件和系统架构允许，尝试并行化初始化操作。比如，同时初始化多个外设，减少总的启动时间。
- **使用引导加载程序（Bootloader）**：将引导加载程序用于加载和初始化主程序。引导加载程序可以进行基本初始化并快速转交给主程序。

在嵌入式系统中，如何处理多核处理器的同步问题？

答案：

- **锁机制**：使用互斥锁、读写锁等机制来保护共享资源，确保多个核心之间的访问不会发生冲突。
- **信号量**：使用信号量来控制对资源的访问，处理核心之间的同步问题。
- **屏障**：使用屏障机制（如全局同步点）来确保所有核心在特定点之前完成其工作，从而协调多核心之间的操作。
- **原子操作**：使用原子操作来保证对共享数据的修改是不可分割的，避免数据竞争。

解释如何实现一个高效的基于事件的异步通信机制。

答案：

- **事件队列**：实现一个事件队列，存储待处理的事件。当事件发生时，将其添加到队列中。
- **事件处理程序**：定义事件处理程序或回调函数，以响应不同类型的事件。事件处理程序会从队列中取出事件并处理。

- **中断驱动**：利用中断来触发事件处理，减少轮询和主动检查的开销。
- **异步处理**：在事件处理程序中执行异步操作，避免阻塞主线程。可以通过多线程或任务调度来实现异步处理。

在嵌入式系统中，如何设计一个可靠的错误检测和恢复机制？

答案：

- **错误检测码**：使用CRC、奇偶校验等技术检测数据传输或存储中的错误。
- **冗余校验**：实现数据冗余机制（如RAID）以确保数据在出现错误时能够恢复。
- **看门狗定时器**：使用看门狗定时器来检测系统是否进入死锁状态，并在必要时重启系统。
- **错误日志**：记录错误信息以便于分析和调试。日志记录应尽可能地高效，避免对系统性能的影响。
- **恢复策略**：定义恢复策略，如重启系统、恢复到安全状态或切换到备用系统。

如何在嵌入式系统中实现一个高效的动态内存分配算法？

答案：

1. **内存池**：分配固定大小的内存块，减少内存碎片。内存池通常预先分配一定数量的块，用于满足动态内存请求。
2. **伙伴系统**：将内存划分为不同大小的块，通过合并和分割操作减少外部碎片。伙伴系统在分配和释放内存时动态调整块的大小。
3. **分配算法**：实现高效的分配算法，如首次适应（First Fit）、最佳适应（Best Fit）和最差适应（Worst Fit）策略，选择适合应用场景的算法。
4. **垃圾回收**：在系统允许的情况下，定期进行垃圾回收，以清理不再使用的内存块。

描述如何在嵌入式系统中实现低功耗操作模式。

答案：

- **睡眠模式**：利用系统的睡眠模式（如浅睡眠、深睡眠）降低功耗。在任务空闲时，将系统或处理器进入低功耗状态。
- **动态电压调整**：根据负载动态调整处理器的电压和频率。降低处理器频率和电压可以减少功耗。
- **关闭外设**：在不需要使用某些外设时，将其关闭以节省功耗。例如，禁用未使用的外设时钟。
- **功耗分析**：使用功耗分析工具来识别和优化系统中的高功耗组件。根据功耗分析结果进行优化。

如何设计一个高效的任务优先级反转解决方案？

答案：

设计高效的任务优先级反转解决方案可以采用以下方法：

- **优先级继承**：实现优先级继承协议，当低优先级任务持有高优先级任务所需的资源时，临时提升低优先级任务的优先级，以避免优先级反转。
- **优先级天花板协议**：为共享资源分配一个最高优先级的任务，当任务请求资源时，资源的优先级被提升到该任务的优先级，避免优先级反转。
- **优先级继承和天花板结合**：结合优先级继承和优先级天花板协议，以提供更强的优先级保护。

解释如何在嵌入式系统中实现快速异常处理。

答案：

- **高效异常处理程序**：编写高效的异常处理程序，确保异常处理尽可能简短。避免在异常处理程序中进行复杂的操作。
- **中断优先级**：配置中断优先级，确保高优先级的中断能够优先处理，减少对系统的影响。
- **中断延迟优化**：减少中断处理的延迟，通过优化中断服务例程（ISR）来提高系统响应速度。
- **异常记录和恢复**：记录异常信息以便于后续分析，并设计合适的恢复策略，如重启或恢复到安全状态。

在嵌入式系统中，如何实现多任务间的有效通信？

答案：

- **消息队列**：使用消息队列传递任务之间的数据和控制信息。消息队列提供了线程安全的通信机制。
- **共享内存**：通过共享内存区域让任务之间直接访问共享数据。需要同步机制（如锁）来防止数据竞争。
- **信号量**：使用信号量来协调任务之间的资源访问，确保任务能够按照预期的顺序执行。
- **事件标志**：使用事件标志来同步任务，任务可以等待特定事件的发生，然后进行相应的处理。

如何处理嵌入式系统中的资源争用问题？

答案：

处理资源争用问题可以采用以下方法：

- **互斥机制**：使用互斥锁、信号量等机制来保护对共享资源的访问。确保只有一个任务可以在任何时刻访问资源。
- **优先级调度**：实现优先级调度机制，确保高优先级任务能够获得资源，避免低优先级任务阻塞高优先级任务。
- **资源分配策略**：设计合理的资源分配策略，避免资源竞争和死锁。使用策略如公平分配和优先级继承来管理资源。

解释如何在嵌入式系统中实现实时数据采集和处理。

答案：

- **实时定时器**：使用高精度定时器定期采集数据，并触发数据采集和处理任务。
- **中断驱动采集**：利用中断驱动的方式进行数据采集，在中断服务例程中读取数据并进行处理。
- **数据处理算法**：实现高效的数据处理算法，以确保数据处理在实时要求内完成。避免在数据处理过程中引入不可预测的延迟。
- **任务调度**：使用实时调度算法（如优先级调度）确保数据采集和处理任务能够及时执行。

如何在嵌入式系统中实现高效的数据加密和解密？

- **选择高效算法**：选择适合嵌入式系统的加密算法，如AES、ChaCha20等，这些算法在性能和安全性方面具有良好平衡。
- **硬件加速**：利用硬件加速（如专用加密模块）提高加密和解密的速度。硬件加速可以显著减少处理开销。
- **算法优化**：优化加密算法的实现，如使用适当的块大小和加密模式，提高加密和解密的效率。

- **数据批量处理**：在处理大量数据时，批量加密和解密数据，减少加密操作的开销。

1. FreeRTOS的启动流程是怎么样的？

1. 初始化阶段：在系统启动时，首先进行系统的基本初始化，包括硬件初始化和内存管理初始化。
2. 创建任务：通过调用 `xTaskCreate`` 或相关函数创建任务。每个任务是 **FreeRTOS** 的基本执行单元。
3. 创建其他系统对象：包括信号量、队列、互斥量等。
4. 启动调度器：通过调用 `vTaskStartScheduler`` 启动 **FreeRTOS** 调度器。调度器将开始根据任务优先级和调度算法调度任务的执行。
5. 任务调度：调度器将按任务的优先级和时间片安排任务执行，任务在运行中可以通过调度器切换。

2.CAN通信协议和I2C、SPI通信协议的比较

CAN (Controller Area Network) :

应用：主要用于汽车和工业自动化领域。

速度：较低（通常高达 **1 Mbps**）。

特性：支持多主机和多从机，具有错误检测和处理机制，支持优先级和仲裁机制。

I2C (InterIntegrated Circuit) :

应用：广泛用于低速外围设备通信，如传感器、EEPROM。

速度：较低（标准模式下最高 **100 kbps**，快速模式下最高 **400 kbps**，快速模式 **Plus** 下最高 **1 Mbps**）。

特性：支持多主机和多从机，使用两条线（**SDA** 和 **SCL**），具有简单的地址选择机制。

SPI (Serial Peripheral Interface) :

应用：用于高速外围设备通信，如闪存、ADC。

速度：较高（可以达到几十 **Mbps**）。

特性：全双工通信，支持多个从设备（需要独立的 **SS** 线），四条线（**MOSI**、**MISO**、**SCK**、**SS**）。

3.FLASH的使用、读和写操作比较

使用：**FLASH** 存储是一种非易失性存储器，用于存储程序代码和配置数据。它在嵌入式系统中广泛使用。

读操作：**FLASH** 读操作相对较快，通常不需要额外的操作时间。

写操作：**FLASH** 写操作较慢，并且通常需要擦除操作来重写数据。写入新的数据之前，必须先擦除相应的存储块。

4.软中断和硬中断的概念

硬中断：由硬件事件触发，通常是外部设备发出的信号。中断处理程序（**ISR**）在硬件中断信号到达时立即响应。

软中断：由软件触发，通常用于实现系统调用或任务调度。它是通过软件指令生成的中断请求，通常用于在操作系统中执行特定操作。

5. ARM架构有哪些异常？

重置异常：系统复位时触发。

未定义指令异常：执行非法指令时触发。

软件中断异常：由 `SWI` 指令触发，通常用于系统调用。

预取异常：当指令预取时发生错误。

数据异常：访问数据时发生错误。

中断请求（**IRQ**）：外部中断。

快速中断请求（**FIQ**）：更高优先级的中断请求。

6. 轮询和中断那个效率高，如果不断触发的情况下呢？

一般情况下：中断通常比轮询更高效，因为它可以在事件发生时立即响应，而轮询需要持续检查状态。

不断触发的情况下：如果中断频繁触发，可能导致中断处理程序频繁执行，从而影响系统性能。在这种情况下，优化中断处理程序，或使用合适的轮询策略来减少中断开销可能更合适。

7. 线程池有了解吗

线程池是一种管理和重用线程的机制。线程池预先创建了一定数量的线程，这些线程可以在任务到来时快速执行。这样可以减少线程创建和销毁的开销，提高系统效率。线程池的核心是一个任务队列和若干工作线程，这些线程不断从任务队列中获取任务并执行。

8. 野指针是如何产生的？

野指针是指向已释放或未初始化内存的指针。它们的产生通常是由于：

内存释放后未置空指针：释放了指针指向的内存后，未将指针设为 `NULL`。

使用未初始化的指针：指针在使用前没有被正确初始化。

数组越界：访问超出数组边界的内存区域。

9. C++的机制是否了解？

面向对象编程：支持类和对象、继承、多态等。

模板：支持泛型编程，可以编写类型安全的代码。

异常处理：使用 `try`、`catch` 和 `throw` 机制处理异常。

运算符重载：允许自定义运算符的行为。

RAII (Resource Acquisition Is Initialization)：通过对象生命周期管理资源（如内存、文件句柄等）。

10. 如果SPI可以去掉一根线，哪一根可以去掉？

在 **SPI** 协议中，如果需要减少线数，可以考虑去掉 **MISO (Master In Slave Out)** 线。然而，这样做会将 **SPI** 从全双工改为半双工模式，只能选择发送数据或接收数据中的一种。

11. 32位系统和64位系统中的32位和64位指的是什么？

32位系统：处理器的数据总线、寄存器和地址总线都是 32 位，能直接寻址的内存空间最大为 4 GB (2^32)。

64位系统：处理器的数据总线、寄存器和地址总线都是 64 位，能直接寻址的内存空间大于 4 GB（通常达到 16 EB），支持更大的内存和更高的计算性能。

12.有没有使用过DMA，与中断的区别是什么？

DMA (Direct Memory Access)：允许外设直接访问内存，而无需 CPU 参与数据传输。它可以显著减少 CPU 的负担，提高数据传输效率。

中断：外设通过中断信号通知 CPU 处理特定事件。中断处理程序在事件发生时被执行，可能会影响 CPU 的正常工作流。

DMA 适用于大量数据传输，能有效减少 CPU 负担，而中断则适用于处理需要立即响应的事件。

13.C 语言中的 enum 类型在不同编译器中的表现可能有什么差异？

存储大小：enum 类型在不同编译器中可能有不同的存储大小。有些编译器将 `enum` 实现为 `int` 类型，而其他编译器可能使用更小的存储空间。

默认值：enum 的默认起始值通常是 0，后续值递增。不同编译器可能对未显式指定值的 `enum` 常量的处理方式有所不同。

类型转换：enum 类型可以转换为整型，但不同编译器在处理 `enum` 和整型之间的转换时可能有不同的行为。需要注意在跨平台编程时，确保代码在所有目标编译器上都能正确编译和运行。

14.如何在多线程环境中正确使用 volatile 关键字？

保证可见性：volatile 确保每次访问变量时都从内存中读取最新的值，而不是从寄存器或缓存中读取。对于共享变量，防止编译器对其进行优化。

与同步机制配合：在多线程环境中，volatile 仅保证变量的可见性，而不提供原子性和同步。通常需要与互斥锁（`mutex`）等同步机制配合使用，以确保线程安全和数据一致性。

15.const和 volatile 关键字结合使用时会发生什么情况？

const：声明变量为常量，编译器将其视为不可修改。常用于定义只读数据。

volatile：声明变量可能会被外部因素改变，禁止编译器优化。常用于硬件寄存器或多线程编程中的共享变量。

结合：将 const 和 volatile 结合使用时，编译器将变量视为可能被外部因素改变的常量，确保每次读取变量时都从内存中读取最新的值，而不是使用缓存。

16.SPI 协议的时序如何影响数据传输的可靠性？

SPI 时序的可靠性由时钟极性（CPOL）和时钟相位（CPHA）决定：

CPOL：时钟极性，定义时钟线的空闲状态是高电平还是低电平。CPOL=0 时，空闲状态为低电平；CPOL=1 时，空闲状态为高电平。

CPHA：时钟相位，定义数据采样和时钟边沿的关系。CPHA=0 表示数据在时钟的上升沿采样，CPHA=1 表示在下降沿采样。

STM32 的启动流程是怎样的？详细描述从上电到进入主函数的过程。

当 STM32 上电时，首先启动的是芯片内部的启动代码。这个代码会先初始化一些关键硬件，比如堆栈指针，然后跳转到复位中断向量，也就是 `Reset_Handler`。在这个处理程序中，系统会进行一些基本的初始化操作，比如设置时钟、初始化内存（包括清空 `.bss` 段、复制 `.data` 段），最后调用 `main()` 函数，这才正式开始执行我们编写的应用程序代码。

描述 STM32 中 NVIC 的工作原理

STM32 中用于管理中断的模块。当外设产生中断时，NVIC 会根据优先级决定是否响应以及何时响应。NVIC 允许多个中断源按照优先级进行嵌套处理，从而确保高优先级的中断能够打断低优先级的中断执行。

如何使用 STM32 的 DMA 实现数据传输？

用于在外设和内存之间快速传输数据的模块，而不需要 CPU 的干预。使用 DMA 时，首先需要配置 DMA 通道，比如设置源地址、目的地址、传输的数据量、传输方向等，然后启动 DMA 通道。之后，DMA 就会自动进行数据传输，当传输完成后，还可以触发一个中断通知 CPU 传输完成了。

STM32 的时钟系统如何配置？请解释 HSE、HSI、PLL 及其用途。

内部时钟（HSI）、外部时钟（HSE）和锁相环（PLL）组成。HSI 是内部振荡器，通常频率较低、精度不高，但适合快速启动和低功耗模式。HSE 是外部晶振，频率高、精度好，适合需要精确时钟的应用。PLL 是一种可以倍频的模块，用于将 HSI 或 HSE 的频率提高，以满足更高的系统时钟需求。

如何在 STM32 中实现 PWM 信号的生成？

生成 PWM 信号通常使用定时器外设。设置定时器的时基单元，确定 PWM 信号的频率，然后配置输出比较单元来设置占空比。通过设置定时器的比较寄存器值，定时器就能在指定的时间点翻转输出，从而产生 PWM 波形。

解释 C 语言中的指针与数组的区别与联系。

数组名在大多数情况下可以看作是指向数组第一个元素的指针，但它不是一个变量，不能修改。而指针则是一个变量，可以指向任何内存地址。通过指针，你可以遍历数组或者访问数组中的元素，但它们之间还是有区别的，比如数组名是一个常量，指针可以被重新赋值。

描述 volatile 关键字的作用，为什么在嵌入式编程中它非常重要？

volatile 告诉编译器不要对声明了这个关键字的变量进行优化。因为在嵌入式系统中，变量的值可能会被外部硬件或中断改变，如果没有 `volatile`，编译器可能会优化掉这些变化，导致程序运行异常。简而言之，`volatile` 确保每次访问该变量时，都会直接从内存读取最新的值。

什么是内存对齐？为什么内存对齐很重要？

内存对齐是指数据在内存中存储时，其地址必须是某个特定的边界值（比如 4 字节或 8 字节）的整数倍。内存对齐非常重要，因为许多处理器在访问未对齐的内存时效率会降低，甚至会引发异常。对齐可以提高内存访问的效率，确保系统的稳定性。

解释 C++ 中的构造函数、析构函数和拷贝构造函数之间的区别。

构造函数是在创建对象时调用的，用于初始化对象。析构函数是在对象销毁时调用的，用于释放资源。拷贝构造函数则是在用一个对象去初始化另一个对象时调用的，它会拷贝对象的内容。简单来说，构造函数是“生”，析构函数是“死”，拷贝构造函数是“复制”。

描述如何在 C++ 中实现多态性，并举例说明。

C++ 中的多态性通常通过继承和虚函数实现。你可以在基类中定义一个虚函数，然后在派生类中重写这个函数。通过基类指针或引用调用这个虚函数时，实际执行的是派生类的版本，这就是多态性。比如，你有一个 `Animal` 基类，其中有一个虚函数 `makeSound()`，在 `Dog` 和 `Cat` 类中重写这个函数，那么 `Animal* a = new Dog(); a->makeSound();` 就会调用 `Dog` 的 `makeSound()`。

什么是静态变量？在嵌入式编程中，静态变量的作用是什么？

静态变量在程序的生命周期内只被初始化一次，并且其值在函数调用结束后依然保持不变。静态变量可以保持某个状态或计数，特别是在资源有限的环境中，它有助于减少内存分配的开销。

如何在 C 中避免内存泄漏？请举例说明常见的内存泄漏场景和防止方法。

避免内存泄漏的关键是确保每个 `malloc` 或 `calloc` 分配的内存都有一个相应的 `free`。常见的内存泄漏场景包括：忘记释放动态分配的内存、循环中频繁分配内存但没有释放、函数内部分配内存后返回前未释放等。为避免这些问题，可以养成在分配完内存后立即编写释放代码的习惯，或者使用智能指针等机制来自动管理内存。

在嵌入式系统中，如何配置和编译一个 Linux 内核？

首先，你需要获取 Linux 内核源代码。然后通过配置工具如 `make menuconfig` 或 `make xconfig`，选择所需的驱动程序和功能。配置完成后，使用 `make` 命令编译内核，并生成内核镜像和模块。最后，将编译好的内核镜像复制到目标设备上，并更新引导程序以使用新内核。

什么是设备树 (Device Tree)？如何在 Linux 中使用它来描述硬件？

设备树是一种用于描述硬件布局的结构化数据。它主要用来在引导时告诉操作系统硬件的具体信息，比如有哪些外设、它们的内存映射、以及连接方式。设备树通常以 `.dts` 文件的形式存在，通过编译生成 `.dtb` 文件，并在内核启动时加载。

如何在 Linux 上编写一个简单的字符设备驱动程序？

首先，编写一个内核模块，定义字符设备的操作函数，如 `open`、`read`、`write` 等。然后，在模块的初始化函数中，使用 `register_chrdev` 函数注册字符设备，并为其分配一个主设备号。

解释 Linux 中进程调度的基本原理，如何在嵌入式 Linux 系统中调优调度器？

Linux 的进程调度器采用了基于优先级的调度策略，调度器会根据每个进程的优先级、运行时间等因素来决定哪个进程应该获得 CPU 使用权。在嵌入式系统中，可以通过调整进程的优先级、使用实时调度策略（如 `SCHED_FIFO`、`SCHED_RR`）来优化调度器，确保关键任务能够按时执行。

在 Linux 环境下，如何实现进程间通信 (IPC)？举例说明管道、消息队列、共享内存等方法的使用。

在 Linux 中，进程间通信（IPC）可以通过多种方式实现，比如管道（pipe）、消息队列（message queue）、共享内存（shared memory）等。管道适合父子进程之间的简单数据传递；消息队列允许不同进程之间通过发送和接收消息来通信；共享内存则提供了多个进程共享同一块内存的方式，适合需要高效数据交换的场景。

解释 Linux 内核模块（LKM）的加载和卸载过程。

Linux 内核模块（LKM）可以动态加载和卸载，不需要重启系统。加载模块时，使用 `insmod` 命令加载模块文件，并自动调用模块的初始化函数。在卸载模块时，使用 `rmmod` 命令，系统会调用模块的清理函数，释放相关资源。

什么是实时操作系统（RTOS）？与普通操作系统相比，有哪些关键区别？

RTOS 是一种保证任务在严格时间约束下执行的操作系统，通常用于需要高实时性和低延迟的场景。与普通操作系统相比，RTOS 更注重任务的确定性调度，而不是最大化系统吞吐量。RTOS 通常提供高优先级任务的快速响应能力，而普通操作系统可能无法保证这一点。

在 RTOS 中，如何管理任务的优先级？请解释优先级倒置以及如何避免它。

在 RTOS 中，任务的优先级决定了调度的顺序。优先级倒置是指低优先级任务占用资源导致高优先级任务被阻塞，从而延迟了高优先级任务的执行。为避免优先级倒置，可以使用优先级继承机制，让低优先级任务在持有资源时临时提高优先级，从而避免高优先级任务的延迟。

如何在 RTOS 中实现任务间的同步与通信？请解释信号量、消息队列、和事件标志的使用。

在 RTOS 中，信号量、消息队列、事件标志是常用的任务同步与通信手段。信号量用于控制对共享资源的访问，防止多个任务同时访问同一资源。消息队列允许任务之间发送和接收消息，是一种方便的通信方式。事件标志则用于任务之间的事件通知，可以让任务等待某些特定事件的发生。

解释 RTOS 中的抢占式调度与合作式调度的区别。

抢占式调度允许高优先级任务随时打断低优先级任务的执行，确保高优先级任务能及时运行。合作式调度则依赖任务自行释放 CPU 控制权，只有当前任务主动放弃 CPU 时，调度器才会切换到其他任务。因此，抢占式调度在实时性和响应速度上更有优势。

如何在 RTOS 中实现一个简单的定时器功能？

在 RTOS 中，定时器功能通常由内置的定时器模块提供。你可以创建一个定时器，并指定定时器的回调函数和触发时间。当时间到达后，RTOS 会自动调用回调函数。定时器可以用于定时任务、延迟操作等场景。

解释 SPI 和 I2C 通信协议的区别及其应用场景。

SPI 是一种高速、全双工的通信协议，通常用于需要高数据传输率的设备间通信，比如显示屏和传感器。I2C 是一种半双工、低速的通信协议，适用于短距离、简单连接的设备，比如传感器、存储器。SPI 的优点是速度快，缺点是需要更多的引脚，而 I2C 的优点是只需要两根线即可实现通信，但速度较慢。

如何在嵌入式系统中实现对 CAN 总线的通信？请描述基本的帧结构和通信流程。

CAN 总线是一种多主模式的串行通信协议，广泛应用于汽车和工业控制领域。要实现 **CAN 总线**通信，你需要配置 **CAN 控制器**，设置波特率、过滤器等。**CAN 帧**的基本结构包括标识符、控制字段、数据字段、**CRC 校验**和帧结尾。当一个节点准备发送数据时，会先发送一个标识符，其他节点根据标识符决定是否接收数据。**CAN 总线**具有较高的抗干扰能力和可靠性。

解释 UART 和 USART 的区别。应用场景是什么？

UART（通用异步收发器）和 **USART**（通用同步异步收发器）主要的区别在于它们支持的通信模式。
UART：只支持异步通信，这意味着没有时钟信号同步数据传输，数据通过起始位和停止位来进行同步。这种方式简单、使用广泛，适用于串口通信，比如在计算机和外部设备之间的连接。
USART：除了支持 **UART** 的异步模式外，还支持同步通信模式。这意味着它可以使用时钟信号与另一台设备同步数据传输。在同步模式下，数据传输速度更快，更适合需要高速度、精确时钟同步的通信，如某些高性能外设的连接。

RTOS 中任务调度的过程及任务上下文切换

任务调度是 **RTOS** 中的核心功能，它决定了哪个任务在某一时刻运行。调度器根据任务的优先级和状态（如就绪、运行、阻塞）来选择下一个要执行的任务。

任务上下文切换是当一个任务停止运行而另一个任务开始运行时，系统需要保存当前任务的状态（上下文），并恢复新任务的状态。这个过程包括：

保存当前任务的上下文（如 **CPU 寄存器**、程序计数器、堆栈指针等）。

选择下一个要运行的任务，并加载该任务的上下文。

切换堆栈，指向新任务的堆栈区域。

跳转到新任务的执行位置，继续执行。

什么是 Bootloader？主要功能是什么？如何实现一个简单的 Bootloader？

是在上电或复位后首先执行的一段程序。初始化硬件，如时钟、内存和外设。检查和加载应用程序代码到内存中。可能会执行一些校验工作，比如校验代码的完整性。

最后，跳转到应用程序的入口点，开始执行用户代码。

思路：

1. 编写初始化代码
2. 编写一段代码从存储器（如 **Flash**）读取应用程序到内存中。
3. 跳转到应用程序的起始地址，开始执行。

Flash 和 EEPROM 的区别及应用场景

Flash：存储密度高、写入速度快，但只能以块（通常是几 **KB**）为单位进行擦除和写入。它通常用于存储固件、操作系统代码或较大数据的存储，比如在 **MCU** 上存储应用程序代码。

EEPROM：可以按字节进行擦除和写入，但写入速度较慢，使用寿命较短。适合需要频繁更新的小数据存储，如配置信息、校准数据等。

应用：

Flash：适合大数据存储，如固件和操作系统镜像。

EEPROM：适合存储少量、需要经常修改的数据，如设备配置和状态。

使用 Watchdog Timer 的作用及设计一个基于 Watchdog 的系统复位机制

是一个定时器，用于检测系统是否正常工作。当系统出现故障或陷入死循环时，可以复位系统。

作用：防止系统卡死：在程序正常运行时，定期重置 **watchdog Timer**。如果在预定时间内没有重置，**watchdog** 会触发系统复位。

定时器有几种用法

- 1.最常用的就是延迟delay
- 2.定时中断，tick中断
- 3.PWM输出控制电机的
- 4.计时计数

数组和链表的区别

- 1.数组要在一块连续的空间上，链表由节点组成。
- 2.链表的内存开销比较大
- 3.数组可以快速访问任意位置的元素，链表只能按顺序访问

队列和栈的区别

- 1.队列是先进先出，栈是先进后出

怎么传输浮点数？

```
void sendFloatAsString(UART_HandleTypeDef *huart, float value) {
    char buffer[20];
    snprintf(buffer, sizeof(buffer), "%f", value);
    HAL_UART_Transmit(huart, (uint8_t*)buffer, strlen(buffer), HAL_MAX_DELAY);
}
```

主设备3.3v从设备1.8v，不使用电平转换如何通过搭电路实现通信

一般高电平的阈值电压是从设备的70%，低电压是主设备的30%，通过降低通信速率，使从设备传输数据是电平数据更加的稳定。

stm32的启动流程

- 1.MCU上电，从0x8000,0000启动(bootloader可以改)
- 2.地址为0x8000004(sp指针偏移+4)处取出复位中断向量的地址
ldr sp, [r0] ; read val from new vector
ldr r1, [r0, #4] ; read val from "new vector + 4"
- 3.复位中断服务程序执行执行main函数死循环

image-20240829201439714

如何对绝对地址0x100000赋值？

c语言
volatile uint32_t* address = (volatile uint32_t*)0x100000;
*address = value;

汇编
LDR R0, =0x100000 ; 将地址0x100000加载到寄存器R0
LDR R1, =value ; 将value加载到寄存器R1
STR R1, [R0] ; 将R1的值存储到R0指向的地址

如何实现缓存？

```

#define FIFO_SIZE 16 // FIFO 缓冲区大小
// 定义 FIFO 结构体
typedef struct {
    uint8_t buffer[FIFO_SIZE];
    uint8_t readIndex;
    uint8_t writeIndex;
} FIFO;
// 初始化 FIFO
void fifoInit(FIFO* fifo) {
    fifo->readIndex = 0;
    fifo->writeIndex = 0;
}
// 向 FIFO 中写入数据
void fifowrite(FIFO* fifo, uint8_t data) {
    uint8_t nextwriteIndex = (fifo->writeIndex + 1) % FIFO_SIZE;
    if (nextwriteIndex != fifo->readIndex) {
        fifo->buffer[fifo->writeIndex] = data;
        fifo->writeIndex = nextwriteIndex;
    }
    else {
        printf("FIFO Overflow\n"); // FIFO 已满，无法写入新数据
    }
}
// 从 FIFO 中读取数据
uint8_t fifoRead(FIFO* fifo) {
    if (fifo->readIndex != fifo->writeIndex) {
        uint8_t data = fifo->buffer[fifo->readIndex];
        fifo->readIndex = (fifo->readIndex + 1) % FIFO_SIZE;
        return data;
    }
    else {
        printf("FIFO Empty\n"); // FIFO 已空，无法读取数据
        return 0; // 返回默认值或错误码
    }
}

```

说一下 volatile，你在什么场景下会使用？

1. 一般我是在全局变量前修饰
2. 用在中断处理函数中的变量，使用到的变量可能会受到中断的干扰，导致变量的值发生变化。
3. 还有我调试的时候有时会加，假如我要记录这个函数跑了多少遍，在这个函数中一个变量++，使用volatile修饰就可以这个变量每次都能加一次，不会因为两次运行的时间实在太短了被cpu优化了，只加了一次。

static修饰的函数的变量、函数以及它的作用举例？

1. 修饰过的东西，限制了作用域，只能在当前文件中用，其他文件用不了，只能通过结构体和extern申明等方式调用
2. 保存上一次函数运行的值，在程序运行的时间内，修饰过的变量初始化只会进行一次。下次函数调用这个变量时还是这个值
3. 举例Linux驱动:修饰过的函数:通过将函数定义为 static，可以隐藏函数的实现细节，只暴露给外部的接口，从而实现信息的封装。这样可以更好地保护函数的内部逻辑，提高代码的安全性。

函数指针是怎么定义的，有什么用？

```
void (*pFunc)(int, int);
```

作用：

1. 回调函数：函数指针可以作为参数传递给另一个函数，以便在需要时回调该函数。
2. 函数的动态调用：函数指针可以根据不同的条件指向不同的函数，以便在运行时动态地调用不同的函数。

中断能否传参

不能

一般都是通过全局变量方式。不过为了避免中断函数重入，我们可以在进入中断服务函数之后将中断关闭，处理完后再开中断。

中断函数是收到中断信号自动跳转的，并非通过函数调用，什么时候运行是预先不确定的，所以不支持传参数。

ADC位数精度如何计算

一个12位的ADC可以将输入的模拟信号转换为一个12位的数字值，即有4096（ 2^{12} ）个离散的量化水平。

位数精度的计算可以通过以下公式进行：

精度 (in bits) = $\log_2$ (量化水平数)

以12位的ADC为例，它具有 $2^{12} = 4096$ 个离散的量化水平。因此，它的位数精度可以通过如下公式计算：

精度 (in bits) = $\log_2 (4096) = 12 \text{ bits}$

对于一个12位ADC，如果其测量范围为0V到3.3V，则最小分辨步进值为：

最小分辨步进值 = $3.3\text{V} / (2^{12}) \approx 0.000805\text{V}$

SPI总线最多可以连接几个设备？为什么是这么多个？

四个，SPI总线的设备数量取决于片选线（CS）的数量。

队列的底层原理

队列存储数据的实质是环形缓冲区

`xQueueCreate()`

1. 在开始创建队列之前，首先要分配内存并检查是否成功。可以使用 `pvPortMalloc()` 函数来分配内存，它是 FreeRTOS 定义的内存分配函数之一。
2. 接着，需要为队列数据结构分配内存。在这个例子中，队列数据结构是一个自定义的 `Queue_t` 结构体，包括队列元素存储区、队列深度、队列互斥量和任务等待列表等。
3. 接下来，初始化队列控制块，包括任务等待列表、队列深度、队列元素大小、读写指针等属性。同时，创建队列的互斥量，用于保护共享资源的互斥访问。
4. 最后，返回队列句柄，并将其用于后续的数据传递和同步等操作。

 image-20240829201705765

`xQueueSend()` 原理

- 1.检查队列是否有空闲空间。
- 2.如果有空闲空间，则将数据拷贝到队列中，并返回 `pdPASS` 表示发送成功。
- 3.如果没有空闲空间，则根据 `xTicksToWait` 参数判断是否等待可用空间：
 - 如果 `xTicksToWait` 为 0，则立即返回错误码 `errQUEUE_FULL`。
 - 如果 `xTicksToWait` 大于 0，则将当前任务添加到等待发送`xTaskWaitingToSend`的任务列表，并将当前任务挂起，让出 CPU 时间片给其他任务执行。
- 4.当有任务从队列接收时，会检查是否有任务等待接收，如果有，则从等待接收的任务列表`xTaskWaitingToReceive`中移除一个任务并激活它。

中断概念，如何中断，硬件中断和软件中断有什么区别，RTOS中的硬中断如何工作，软中断如何工作

- 1、软件中断由RTOS API调用或设置相应的标志位来提出中断请求/软件中断的响应时间相对较慢，因为在触发软件中断后，需要进行上下文切换到内核模式，并执行中断处理函数/软件中断处理函数在内核模式下执行。它们可以访问受保护的系统资源，并执行更复杂的操作/软件中断有优先级
- 2、硬件中断由系统外部的硬件设备或外设触发/硬件中断通常具有固定的响应时间，因为它们与硬件设备直接相关，且在实际硬件设计中有严格的时序要求/硬件中断处理函数在硬件中断上下文中执行。它们通常需要快速完成，并尽可能少地干扰其他任务的执行/硬件中断具有固定的优先级，通常由硬件设备决定。在大多数情况下，硬件中断的优先级比任务调度器的优先级高。

软件中断的处理过程

- 1.中断请求，RTOS API调用或设置相应的标志位来提出中断请求
- 2.当中断请求被触发时，RTOS 将执行上下文切换，从用户模式切换到内核模式。
- 3.中断处理函数执行，在内核模式下，RTOS 将调用与软中断相关联的中断处理函数
- 4.返回到任务执行：当中断处理完成后，RTOS 将执行上下文切换，从内核模式切换回用户模式。

RTOS系统运行中硬中断发生时，RTOS系统会如何处理

- 1.发生硬件中断
- 2.响应中断，rtos系统暂停任务，保存当前任务的上下文
- 3.中断程序执行
- 4.中断程序完成，恢复现场
- 5.任务继续执行

你是如何调试的？

- 1.发生硬件中断
- 2.响应中断，rtos系统暂停任务，保存当前任务的上下文
- 3.中断程序执行
- 4.中断程序完成，恢复现场
- 5.任务继续执行

有名管道和无名管道

无名管道对于进程是有关系的，数据在管道中按照先进先出（FIFO）的顺序传递
有名管道对于进程没有要求，任意进程都可以通过打开同一个名字的有名管道来进行通信

如何进行内核同步

在 Linux 中，内核同步是指确保多个进程或线程在访问共享资源时的正确性和一致性。下面是一些常用的内核同步机制：

1. 互斥锁（**Mutex**）：互斥锁是一种简单的同步机制，用于保护临界区（即共享资源）。只有一个线程可以获得该锁，并且其他线程必须等待。可以使用 `pthread_mutex_*` 函数或 Linux 的 `mutex_lock()` 和 `mutex_unlock()` 系统调用来操作互斥锁。

2. 读写锁（**Read-Write Lock**）：读写锁用于实现共享资源的读写操作。多个线程可以同时拥有读锁，但只有一个线程可以拥有写锁。读写锁提供了更高的并发性能，适合读取频繁、写入较少的场景。可以使用 `pthread_rwlock_*` 函数或 Linux 的 `rwlock_*` 系统调用来操作读写锁。

3. 条件变量（**Condition Variable**）：条件变量用于在线程之间进行通信，允许线程等待某个条件为真。当条件不满足时，线程可以阻塞等待，直到其他线程通过发送信号或广播唤醒它们。可以使用 `pthread_cond_*` 函数或 Linux 的 `pthread_cond_*` 系统调用来操作条件变量。

4. 信号量（**Semaphore**）：信号量是一种计数器，用于控制对共享资源的访问。它可以用来限制同时访问资源的线程数量。通过原子操作对信号量进行增加或减少，线程可以获取或释放信号量。可以使用 `sem_*` 函数或 Linux 的 `sem_*` 系统调用来操作信号量。

5. 自旋锁（**Spinlock**）：自旋锁是一种忙等待的锁机制，适合于临界区很短，线程获取锁的时间较短的场景。当线程尝试获取锁时，如果锁已被占用，线程会一直循环忙等待，直到锁被释放。可以使用 `spin_lock()` 和 `spin_unlock()` 函数来操作自旋锁。

linux的中断实现

1. 初始化: 设置中断源，让它可以设置中断，设置中断控制器(屏蔽、优先级); 设置CPU总开关(使能中断)
2. 执行程序
3. 产生中断
4. CPU每执行完一条指令就检查有无
5. 发现中断，对于不同的中断，跳去不同的地址执行。

如果再次发生了中断，那么下半部被打断，先执行硬件中断的处理函数: 硬件中断处理完发现“之前已经在处理软件中断”了，就不会往下“再次开始新的处理软件中断流程”而是退出中断处理、并恢复原来的工作: 继续执行刚刚被打断的“软件处理函数”。

为什么启动时关闭icache和dcache?

启动时关闭指令缓存（**icache**）和数据缓存（**dcache**）是为了确保代码的正确执行和数据的一致性。以下是一些可能的原因：

避免缓存脏数据：在系统启动过程中，可能存在一些临时数据或代码片段被写入到缓存中。如果这些数据不是来自可靠的来源或者没有正确初始化，那么在启动后读取这些缓存中的数据可能会导致错误的结果。关闭缓存可以确保在启动阶段使用的是内存中的最新数据。

避免指令重排序：现代处理器为了提高执行效率，可能会对指令进行乱序执行或重排序。在启动阶段，指令的执行顺序可能非常重要，特别是在初始化硬件和引导加载程序时。关闭指令缓存可以确保指令按照正确的顺序执行，避免因指令重排序而导致的错误。

linux 驱动里面的 probe 函数是什么时候被调用以及它的作用

当一个设备被插入到系统中，并且与相应的驱动匹配成功时，内核会调用该驱动的 `probe` 函数

probe函数的作用：

1. 获取硬件信息，在字符型驱动程序中，一般使用 `gpio_get->` 设置 `gpio` 的模式 `input/output(gpio_direction_input/gpio_direction_output)`

我自己的话写驱动的话，我的中断也是在 `probe` 函数中申请的。

2. 注册设备 `device_create`

FreeRTOS的启动流程是怎么样的？

1. 初始化阶段：在系统启动时，首先进行系统的基本初始化，包括硬件初始化和内存管理初始化。
2. 创建任务：通过调用 `xTaskCreate` 或相关函数创建任务。每个任务是 **FreeRTOS** 的基本执行单元。
3. 创建其他系统对象：包括信号量、队列、互斥量等。
4. 启动调度器：通过调用 `vTaskStartScheduler` 启动 **FreeRTOS** 调度器。调度器将开始根据任务优先级和调度算法调度任务的执行。
5. 任务调度：调度器将按任务的优先级和时间片安排任务执行，任务在运行中可以通过调度器切换。

CAN通信协议和I2C、SPI通信协议的比较

CAN (Controller Area Network) :

应用：主要用于汽车和工业自动化领域。

速度：较低（通常高达 **1 Mbps**）。

特性：支持多主机和多从机，具有错误检测和处理机制，支持优先级和仲裁机制。

I2C (InterIntegrated Circuit) :

应用：广泛用于低速外围设备通信，如传感器、EEPROM。

速度：较低（标准模式下最高 **100 kbps**，快速模式下最高 **400 kbps**，快速模式 **Plus** 下最高 **1 Mbps**）。

特性：支持多主机和多从机，使用两条线（**SDA** 和 **SCL**），具有简单的地址选择机制。

SPI (Serial Peripheral Interface) :

应用：用于高速外围设备通信，如闪存、ADC。

速度：较高（可以达到几十 **Mbps**）。

特性：全双工通信，支持多个从设备（需要独立的 **SS** 线），四条线（**MOSI**、**MISO**、**SCK**、**SS**）。

FLASH的使用、读和写操作比较

使用：**FLASH** 存储是一种非易失性存储器，用于存储程序代码和配置数据。它在嵌入式系统中广泛使用。

读操作：**FLASH** 读操作相对较快，通常不需要额外的操作时间。

写操作：**FLASH** 写操作较慢，并且通常需要擦除操作来重写数据。写入新的数据之前，必须先擦除相应的存储块。

软中断和硬中断的概念

硬中断：由硬件事件触发，通常是外部设备发出的信号。中断处理程序（**ISR**）在硬件中断信号到达时立即响应。

软中断：由软件触发，通常用于实现系统调用或任务调度。它是通过软件指令生成的中断请求，通常用于在操作系统中执行特定操作。

定时器有几种用法

数组和链表的区别

队列和栈的区别

怎么传输浮点数？

主设备3.3v从设备1.8v，不使用电平转换如何通过搭电路实现通信

stm32的启动流程

如何对绝对地址0x100000赋值？

如何实现缓存？

说一下 volatile，你在什么场景下会使用？

static修饰的函数的变量、函数以及它的作用举例？

函数指针是怎么定义的，有什么用？

中断能否传参

ADC位数精度如何计算

SPI总线最多可以连接几个设备？为什么是这么多个？

RTOS队列的底层原理

xQueueSend() 原理

中断概念，如何中断，硬件中断和软件中断有什么区别，RTOS中的硬中断如何工作，软中断如何工作

RTOS系统运行中硬中断发生时，RTOS系统会如何处理

你是如何调试的？

有名管道和无名管道

如何进行内核同步

linux的中断实现

为什么启动时关闭icache和dcache？

linux 驱动里面的 probe 函数是什么时候被调用以及它的作用

CAN通信协议和I2C、SPI通信协议的比较

FreeRTOS的启动流程是怎么样的？

FLASH的使用、读和写操作比较

软中断和硬中断的概念

Linux上的异步IO

在 **Linux** 系统里，异步 **I/O** 让程序在等待数据读写时，能继续做其他事情，这样可以提高效率。它主要用三种方法来实现：**select**、**poll** 和 **epoll**。

select、poll 和 epoll

select：这是最早的方式，用来检查多个文件描述符（比如网络连接）哪个准备好进行操作。它有个限制，就是最多只能处理 **1024** 个文件描述符。

poll：比 **select** 更灵活，不受文件描述符数量的限制。但它的效率不高，因为每次都要遍历所有文件描述符。

epoll：这是针对高并发的场景设计的，效率更高。**epoll** 只在文件描述符真的有事件发生时才通知，大大提高了处理效率。特别适合大量连接的情况。

多线程的参与

这些 **I/O** 方法本身不涉及多线程，不过你可以在多线程程序中使用它们。例如，一个线程专门处理 **I/O** 事件，而其他线程处理不同的任务。

自定义的通信协议

自定义通信协议就是你根据需要设计数据交换规则。

消息格式：怎么设计数据包的结构。

数据编码：如何把数据转成字节流。

错误处理：如何应对通信中出现的问题。

流量控制：如何避免数据传输过快造成拥塞。

Linux中一个socket的内存消耗

协议栈：例如，**TCP** 的控制块和缓冲区会占用内存。

文件描述符：每个 **socket** 会占用一个文件描述符。

应用层缓冲区：应用程序可能为 **socket** 设置额外的内存。

通常每个 **socket** 的内存消耗在几百KB到几MB之间，具体情况取决于系统配置和使用的协议。

TCP建立长连接

TCP 建立长连接的过程叫做“三次握手”：

SYN：客户端发出请求，表示想要建立连接。

SYN-ACK：服务器回应，确认收到请求。

ACK：客户端再确认一次，连接建立成功。

建立了连接后，双方就可以开始数据传输了。

Linux的文件系统

目录结构：文件按树状结构组织。

文件系统类型：有很多种，比如 **ext4**、**XFS**、**Btrfs** 等。

权限管理：可以设置文件的读、写、执行权限。

文件系统加载过程

引导加载程序（**Bootloader**）：在系统启动时，**Bootloader**（像 **GRUB**）会加载 **Linux** 内核。

内核启动：内核会初始化硬件，并挂载根文件系统。

文件系统初始化：内核识别并挂载文件系统，开始管理数据。

用户空间初始化：系统启动用户空间的程序，比如 **init** 进程，进一步初始化系统服务和用户环境。

死锁，怎么生成的，举个例子

死锁 是一种程序错误状态，发生在两个或多个进程（或线程）互相等待对方释放资源，结果大家都陷入了无限等待中，程序就卡住了。

生成死锁的条件：

互斥：资源只能由一个进程占用，比如一个打印机一次只能被一个进程使用。

持有并等待：一个进程已经占有某个资源，并且还在等待其他进程释放其他资源。

非抢占：已经占用的资源不能被强制回收，只能等进程自己释放。

循环等待：形成一个进程的环，每个进程等待下一个进程持有的资源。

虚拟地址和物理地址

虚拟地址 是进程使用的地址，物理地址 是实际内存中数据的真实位置。虚拟地址和物理地址之间的关系是通过内存管理单元（**MMU**）和页表来建立的。

如何建立关系：

虚拟地址：程序运行时使用虚拟地址，这些地址是进程认为的地址。

页表：内核维护一个页表，记录虚拟地址和物理地址的映射关系。

MMU：内存管理单元将虚拟地址转换为物理地址，查找页表来找到数据在物理内存中的实际位置。

多线程和多进程的区别

多线程 和 多进程 都是并发执行任务的方式，但它们有一些关键的不同点：

多进程：

每个进程有独立的内存空间。

进程间通信（**IPC**）比较复杂，因为它们不能直接共享数据。

如果一个进程崩溃了，其他进程不会受到影响。

比较适合需要隔离和稳定性的任务。

多线程：

线程在同一个进程中，共享内存空间。

线程间通信比较简单，因为它们可以直接访问共享数据。

一个线程崩溃可能会影响到整个进程。

比较适合需要共享数据和资源的任务

手撕自守数

```
1^1 = 1
2^2 = 4
3^3 = 27
// 函数来检查是否是自守数
bool is_self_power_number(int n) {
    // 计算 n^n
    long long result = 1;
    for (int i = 0; i < n; i++) {
        result *= n;
    }

    // 转换成字符串
    char result_str[100];
```

```

    sprintf(result_str, "%lld", result);

    char n_str[10];
    sprintf(n_str, "%d", n);

    // 获取结果字符串的长度
    int result_len = strlen(result_str);
    int n_len = strlen(n_str);

    // 检查结果的最后 n_len 位是否与 n 相同
    if (result_len >= n_len) {
        return strcmp(result_str + result_len - n_len, n_str) == 0;
    }

    return false;
}

```

Linux上的异步IO了解过吗?

讲一下select/poll/epoll

这中间有多线程的参与吗?

讲一下自定义的通信协议

Linux中一个socket大概有多少内存消耗?

TCP要建立一个长连接，是怎么做的?

Linux的文件系统能描述一下吗

从Linux kernel启动开始，介绍一下文件系统和整个加载的流程

问死锁，怎么生成的，举个例子?

进程的虚拟地址有多少，物理地址和虚拟地址的关系，这两种地址是怎么建立起关系的?

多线程多进程的区别?

1. I2C 地址

I2C 设备地址用于在 I2C 总线上唯一标识一个设备。通常，它是 7 位或 10 位长。在通信中，地址和读/写位（1 位）一起构成一个完整的 8 位数据。I2C 地址的格式通常是：

```
| 7-bit Address | R/W |
```

其中 7-bit Address 是设备地址，R/W 表示读/写操作。设备手册会提供具体的地址信息。

2. ADC 电池采样电压处理公式

ADC（模拟到数字转换器）用于将模拟电压信号转换为数字值。假设你有一个 12 位的 ADC，电池的电压信号范围是 0 到 3.3V。采样到的数字值可以通过以下公式转换为实际电压值：

$$V_{battery} = \left(\frac{ADC_value}{ADC_max} \right) \times V_{ref}$$

($V_{battery}$) 是电池电压。

(ADC_value) 是 ADC 读数。

(ADC_max) 是 ADC 的最大值，对于 12 位 ADC，通常是 4095。

(V_{ref}) 是 ADC 的参考电压。

3. 设计一个 FIFO

FIFO（先进先出）队列是一种数据结构，遵循先进先出的原则。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define SIZE 10 // 队列大小

typedef struct {
    int data[SIZE];
    int front, rear, count;
} FIFOQueue;

void initQueue(FIFOQueue *q) {
    q->front = 0;
    q->rear = 0;
    q->count = 0;
}

bool isFull(FIFOQueue *q) {
    return q->count == SIZE;
}

bool isEmpty(FIFOQueue *q) {
    return q->count == 0;
}

void enqueue(FIFOQueue *q, int value) {
    if (isFull(q)) {
```

```

        printf("Queue is full\n");
        return;
    }
    q->data[q->rear] = value;
    q->rear = (q->rear + 1) % SIZE;
    q->count++;
}

int dequeue(FIFOQueue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return -1; // Error value
    }
    int value = q->data[q->front];
    q->front = (q->front + 1) % SIZE;
    q->count--;
    return value;
}

int main() {
    FIFOQueue q;
    initQueue(&q);

    enqueue(&q, 1);
    enqueue(&q, 2);

    printf("Dequeued: %d\n", dequeue(&q));
    printf("Dequeued: %d\n", dequeue(&q));

    return 0;
}

```

4. 多线程的注意事项

在多线程项目中，以下几点是关键：

- **线程同步**：使用互斥锁（mutexes）和条件变量（condition variables）来防止数据竞争。
- **死锁避免**：确保锁的获取顺序一致，避免环形等待。
- **资源共享**：避免线程间对共享资源的冲突访问。
- **线程安全**：保证代码在多线程环境下能够正确执行。
- **性能优化**：避免过多线程创建和销毁，使用线程池等技术。

5. 指针

指针是一个变量，用于存储另一个变量的内存地址。在 C 语言中，指针用于动态内存分配、数组和字符串操作、函数参数传递等。以下是一些常见的指针操作示例：

```
int main() {
    int value = 10;
    int *ptr = &value; // 指针指向变量 value 的地址

    printf("Value: %d\n", value);
    printf("Pointer value: %p\n", (void*)ptr);
    printf("Dereferenced pointer value: %d\n", *ptr);

    return 0;
}
```

6. Linux 编程和 Linux 指令

Linux 编程通常涉及系统调用、文件操作、进程管理等。常用的 Linux 指令包括：

- `ls`：列出目录内容
- `cd`：更改目录
- `cp`：复制文件
- `mv`：移动文件
- `rm`：删除文件
- `grep`：搜索文本
- `ps`：查看进程状态
- `top`：实时查看系统资源

7. STM32 启动流程

STM32 启动流程如下：

复位：上电后，处理器复位。

启动引导加载程序（Bootloader）执行内置的引导程序。

初始化硬件：设置时钟、外设等。

跳转到应用程序：启动主应用程序的入口点。

8. STM32 寄存器的作用

- **GPIO 寄存器**：配置和控制 GPIO 引脚。
- **USART 寄存器**：配置和控制串口通信。
- **TIM 寄存器**：配置和控制定时器。
- **ADC 寄存器**：配置和控制模拟到数字转换器。

9. SPI 调试

SPI（串行外设接口）是一种用于设备间通信的协议。如果你使用示波器调试 SPI，可以：

- 配置 SPI 设备的参数（如时钟极性、相位）。
- 通过 SPI 发送和接收数据。
- 使用示波器或逻辑分析仪查看 SPI 信号的波形。

10. 手撕链表翻转

```
#include <stdio.h>
#include <stdlib.h>
```

```

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void printList(Node *head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

Node* reverseList(Node *head) {
    Node *prev = NULL;
    Node *current = head;
    Node *next = NULL;

    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    return prev;
}

int main() {
    Node *head = (Node*)malloc(sizeof(Node));
    head->data = 1;
    head->next = (Node*)malloc(sizeof(Node));
    head->next->data = 2;
    head->next->next = (Node*)malloc(sizeof(Node));
    head->next->next->data = 3;
    head->next->next->next = NULL;

    printf("Original list:\n");
    printList(head);

    head = reverseList(head);

    printf("Reversed list:\n");
    printList(head);

    return 0;
}

```

11. stm32是大端还是小端？手撕一个判断大端还是小端的程序
使用小端字节序。

```

#include <stdio.h>

int isLittleEndian() {
    unsigned int x = 1;
    char *c = (char*)&x;

```

```
        return (*c == 1);
    }

    int main() {
        if (isLittleEndian()) {
            printf("系统是小端字节序\n");
        } else {
            printf("系统是大端字节序\n");
        }
        return 0;
    }
}
```

如果结果是 1，说明系统是小端字节序。

volatile关键字作用

volatile 是用于指示编译器，一个变量的值可能会被程序之外的因素改变，因此禁止编译器对该变量进行优化。通常用于多线程编程中，确保线程间共享的变量的可见性和内存一致性。具体作用包括：
防止编译器对该变量进行优化，确保每次访问该变量时，都是直接从内存中读取，而不是使用寄存器缓存。

代码题：两个升序链表合并成一个降序链表

```
#include <stdio.h>
#include <stdlib.h>

struct ListNode {
    int val;
    struct ListNode *next;
};

struct ListNode* create_node(int val) {
    struct ListNode* new_node = (struct ListNode*)malloc(sizeof(struct
ListNode));
    new_node->val = val;
    new_node->next = NULL;
    return new_node;
}

struct ListNode* mergeTwoLists(struct ListNode* l1, struct ListNode* l2) {
    struct ListNode dummy;
    struct ListNode* current = &dummy;
    dummy.next = NULL;

    // 合并两个升序链表
    while (l1 != NULL && l2 != NULL) {
        if (l1->val < l2->val) {
            current->next = l1;
            l1 = l1->next;
        } else {
            current->next = l2;
            l2 = l2->next;
        }
        current = current->next;
    }

    current->next = (l1 != NULL) ? l1 : l2;

    // 将合并后的链表逆序
    struct ListNode* prev = NULL;
    struct ListNode* next = NULL;
    current = dummy.next;

    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
```

```

    }

    return prev;
}

void printList(struct ListNode* head) {
    while (head != NULL) {
        printf("%d -> ", head->val);
        head = head->next;
    }
    printf("NULL\n");
}

void freeList(struct ListNode* head) {
    struct ListNode* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    struct ListNode* l1 = create_node(1);
    l1->next = create_node(3);
    l1->next->next = create_node(5);

    struct ListNode* l2 = create_node(2);
    l2->next = create_node(4);
    l2->next->next = create_node(6);

    // 合并两个链表并将其逆序
    struct ListNode* merged_list = mergeTwoLists(l1, l2);

    // 打印合并后的链表
    printList(merged_list);

    // 释放链表内存
    freeList(merged_list);

    return 0;
}

```

将无符号 `int` 类型数的二进制数最后一位由 1 改成 0

```

unsigned int num = 5; // 示例值: 101
num = num & (~1);    // 清除最后一位

```

UART 的数据格式? 如何保证在侦听 UART 时不会一直判断接收到 0?

起始位（1 位）：表示数据传输的开始，通常为低电平 `0`。

数据位（5 至 9 位）：实际的数据位，可以配置为 5 到 9 位。

奇偶校验位（1 位，可选）：用于检测传输数据的奇偶性。

停止位（1 至 2 位）：表示数据传输的结束，通常为高电平 `1`。

配置硬件中断：当 UART 接收到数据时触发中断，避免主动轮询判断。

设置超时机制：如果长时间未接收到数据，可以认为通信中断或数据错误。

检查起始位：UART 在接收数据前通常会检测到起始位的下降沿，即 `1` 到 `0` 的转换，这可以避免误判接收到的 0。

寄存器长度变小，寻址范围会变小么？

寄存器长度变小会导致寻址范围变小。

寄存器长度：指的是寄存器的宽度（通常以位为单位，如 32 位、64 位）。

寻址范围：通常由寄存器的宽度决定。例如，32 位的寄存器可以寻址的范围是 2^{32} ，即 4GB 地址空间；而 64 位的寄存器可以寻址的范围是 2^{64} 。

如果寄存器长度变小，能表示的地址空间也会相应减小，从而导致可寻址范围变小。

中断的上半部与下半部是如何分离实现的

上半部

上半部是中断处理的第一部分，通常在中断发生时立即执行，以下是其主要特点：

快速响应：上半部的主要任务是快速响应中断并执行一些时间紧迫的操作，例如确认中断源、保存现场信息、清除中断标志等。

在中断上下文执行：上半部在中断上下文中执行，也就是说，它是在处理器中断时运行的。这意味着在执行上半部时，系统无法处理其他中断（除非支持嵌套中断）。

不进行耗时操作：由于上半部在中断上下文中执行且影响系统的整体响应性，因此通常只执行那些必须立即处理的任务，避免任何可能导致延迟的操作，如访问慢速设备或进行复杂计算。

下半部

下半部是中断处理的延续部分，主要用于处理那些可以稍后完成的操作。它的主要特点包括：

延迟执行：下半部的任务可以延迟执行，通常在操作系统恢复到正常调度状态后执行。这意味着它是在内核进程上下文中执行的，不是在中断上下文中。

处理耗时操作：由于下半部不在中断上下文中执行，它可以进行更多的耗时操作

为什么要分离上半部和下半部？

提高响应性：中断上半部处理时间越短，系统对其他中断的响应速度就越快。因此，必须立即执行的操作放在上半部，而可以稍后处理的任务则放在下半部。

降低中断处理对系统的影响：如果所有中断处理都在中断上下文中进行，可能会导致系统其他重要任务被长期阻塞，影响整体性能。通过分离，将较复杂和耗时的任务放到下半部处理，可以减少这种影响

字符驱动的poll机制

wq是一个等待队列，poll机制只是将线程放进wq中，但是未休眠
for会执行两次，第一次查询没有数据，休眠一段时间，第二次查询，没有数据返回，假如有数据按键按下了，在按键中断程序将event=tured; wake_up_interruptible(wq);
我们只用实现挂入队列和返回状态，剩下的内核帮你实现了

app怎么找到驱动?

通过属性(是什么驱动程序)、主设备号、次设备号
某种程度下讲，/dev/driver它的属性会放在一个字符设备链表，sys_open就会在这个链表 里面找chrdevs[主设备号]这一项

自旋锁、信号量和互斥量三个的区别

1. 自旋锁: 自旋锁也和信号量差不多，不过自旋锁获得锁之前会忙等待，cpu无法执行其他的事情，不断的查询这个锁的状态
2. 信号量: 信号量访问的时候就是申请使用，能不能支持多个线程同时访问这个信号资源，要在信号量初始化sem_init的第二个参数设置，然后释放信号量
3. 互斥量: 互斥量就是一个特殊的信号量，互斥只能一个线程持有，一个线程获得锁之后，其他线程只能等待这个线程释放锁

单核和多核情况下的自旋锁的区别

单核情况下: 假如一个任务要获得自旋锁，它就一直查询，但是cpu只有一个，其他的任务它无法执行，那么就无法释放自旋锁，那么就一直卡死
多核情况: 可以正常执行，一个核忙等待，一个核继续处理其他事情

linux 驱动里面的 probe 函数是什么时候被调用以及它的作用

当一个设备被插入到系统中，并且与相应的驱动匹配成功时，内核会调用该驱动的 probe 函数
probe函数的作用：
1. 获取硬件信息，在字符型驱动程序中，一般使用gpiod_get->设置gpio的模式
input/output(gpiod_direction_input/gpiod_direction_output)
我自己的话写驱动的话，我的中断也是在probe函数中申请的。
2. 注册设备device_create

为什么栈从上往下生长

堆是往上涨的，栈是往下涨的，假如栈往上涨的话，那么要确定堆和栈的分界线，每一个程序的堆和栈都一样，所以这个分界线不好确定，所以一个往上涨，一个往下长可以最大化的利用内存的空间